XCAGI 企业 AI 员工平台

程序鉴别材料 - 源代码文档（V10.0.0）

本材料包含软件源代码前 30 页与后 30 页，共 60 页。
著作权人：李佳泷
开发完成日期：2026 年 5 月

```
================================================================================
XCAGI 企业 AI 员工平台 V10.0.0 - 源代码文档
软件著作权申请专用
生成时间：2026-06-25 03:38:27
================================================================================
================================================================================
文件：前端源代码.txt
行数：171253
================================================================================
// 文件：App.vue
<script setup>
import MainLayout from './components/MainLayout.vue'
import LegacyFloatPanels from '@/components/shell/LegacyFloatPanels.vue'
import AppGlobalProviders from '@/components/shell/AppGlobalProviders.vue'
import VirtualCursorOverlay from '@/components/aiopen/VirtualCursorOverlay.vue'
import { useAppBoot } from '@/composables/useAppBoot'
const {
hideChrome,
appReady,
isProMode,
handleToggleProMode,
isAdminConsoleSpa,
} = useAppBoot()
</script>
<template>
<div class="app-shell" :class="{ 'is-ready': appReady || hideChrome, 'app-shell--bare': hideChrome }">
<LegacyFloatPanels v-if="!hideChrome" />
<AppGlobalProviders :show-lan-gate="!isAdminConsoleSpa()" />
<VirtualCursorOverlay />
<router-view v-if="hideChrome" />
<MainLayout
v-else
:is-pro-mode="isProMode"
@toggle-pro-mode="handleToggleProMode"
>
<router-view v-slot="{ Component, route }">
<transition name="route-fade" mode="out-in">
<div :key="route.fullPath" class="route-view-shell">
<keep-alive v-if="!isAdminConsoleSpa()" :max="12">
<component :is="Component" />
</keep-alive>
<component v-else :is="Component" />
</div>
</transition>
</router-view>
</MainLayout>
</div>
```

```
</template>
<style>
.app-shell {
opacity: 1;
transition: opacity 320ms ease;
height: 100vh;
overflow: hidden;
background:
radial-gradient(circle at 50% 0%, rgba(255, 255, 255, 0.88), transparent 42%),
linear-gradient(135deg, #edf5fb 0%, #e7eef6 48%, #eef3f8 100%);
}
@media (prefers-reduced-motion: reduce) {
.app-shell {
transition-duration: 1ms;
}
}
.app-shell.app-shell--bare {
min-height: 100dvh;
display: flex;
flex-direction: column;
}
.app-shell.app-shell--bare > :last-child {
flex: 1 1 auto;
min-height: 0;
width: 100%;
}
.route-view-shell {
flex: 1 1 auto;
min-height: 0;
display: flex;
flex-direction: column;
overflow: hidden;
}
.route-view-shell > * {
flex: 1 1 auto;
min-height: 0;
}
.route-fade-enter-active,
.route-fade-leave-active {
transition: opacity 250ms ease;
}
.route-fade-enter-from,
.route-fade-leave-to {
opacity: 0;
}
@media (prefers-reduced-motion: reduce) {
.route-fade-enter-active,
.route-fade-leave-active {
transition-duration: 1ms;
}
```

```
}
</style>
// 文件：main.ts
import { createApp } from 'vue';
import { createPinia } from 'pinia';
import * as ElementPlusIconsVue from '@element-plus/icons-vue';
import './fhd/installFetchDbReadToken';
import { useAppShellStore } from '@/stores/appShell';
import {
bootstrapAdminConsoleShellDefaults,
bootstrapEditionDefaults,
bootstrapEnterpriseShellDefaults,
} from '@/constants/platformShellMode';
bootstrapEditionDefaults();
bootstrapEnterpriseShellDefaults();
bootstrapAdminConsoleShellDefaults();
import App from './App.vue';
import router from './router';
import { bindTutorialRouter } from '@/stores/tutorial';
import { registerAllModRoutesFromGlob, registerModRoutes } from './router/registerModRoutes';
bindTutorialRouter(router);
import { fetchModRoutesPayloadShared } from './utils/modRoutesSharedFetch';
import { CLIENT_MODS_UI_OFF_KEY } from '@/stores/mods';
import { applySidebarThemeFromStorage } from './utils/sidebarTheme';
import { installClientConsoleBridge } from './utils/clientDebugLog';
import { initServiceWorker, unregisterStaleServiceWorkers } from './utils/serviceWorker';
import { bootstrapHostConfig } from '@/stores/hostConfig';
import './styles/tokens.css';
import './styles/css/base.css';
import './styles/css/components/sidebar.css';
import './styles/css/components/chat.css';
import './styles/css/components/tables.css';
import './styles/css/components/modals.css';
import './styles/css/components/ui-components.css';
import './styles/css/animations/transitions.css';
import './styles/css/animations/ui-effects.css';
import './styles/css/animations/pro-mode.css';
import './styles/css/animations/gpu-optimizations.css';
import './styles/css/office-theme.css';
import 'font-awesome/css/font-awesome.min.css';
function readVanillaNoModUi(): boolean {
try {
return localStorage.getItem(CLIENT_MODS_UI_OFF_KEY) === '1';
} catch {
return false;
}
}
function browserFullPath(): string {
if (typeof window === 'undefined' || !window.location) return '/';
```

```
const base = String(import.meta.env.BASE_URL || '/').replace(/\/$/, '');
let pathname = window.location.pathname || '/';
if (base && base !== '/' && pathname.startsWith(`${base}/`)) {
pathname = pathname.slice(base.length) || '/';
} else if (base && base !== '/' && pathname === base) {
pathname = '/';
}
return `${pathname}${window.location.search || ''}${window.location.hash || ''}`;
}
function scheduleRouterAddressSync(reason: string): void {
if (typeof window === 'undefined') return;
const syncOnce = async () => {
const target = browserFullPath();
if (!target || target === router.currentRoute.value.fullPath) return;
const resolved = router.resolve(target);
if (!resolved.matched.length) return;
try {
await router.replace(target);
} catch (e) {
if (import.meta.env.DEV) {
console.warn(`[bootstrap] route address sync failed (${reason}):`, e);
}
}
};
void syncOnce();
window.setTimeout(() => {
void syncOnce();
}, 500);
window.setTimeout(() => {
void syncOnce();
}, 1500);
}
async function bootstrap() {
bootstrapEditionDefaults();
void bootstrapHostConfig();
applySidebarThemeFromStorage();
installClientConsoleBridge();
if (typeof window !== 'undefined') {
const host = window.location.hostname;
const port = window.location.port;
const isLocalHost =
import.meta.env.DEV || host === '127.0.0.1' || host === 'localhost';
if (isLocalHost) {
void unregisterStaleServiceWorkers();
} else {
initServiceWorker()
.then((swStatus) => {
if (swStatus.registered) {
console.log(`[Bootstrap] Service Worker ready (offline: ${swStatus.offline})`);
}
```

```
})
.catch(() => {});
}
}
const app = createApp(App);
const pinia = createPinia();
app.config.errorHandler = (err, _instance, info) => {
console.error('[Vue]', info || 'unknown hook', err);
};
window.addEventListener('unhandledrejection', (event) => {
console.error('[unhandledrejection]', event.reason);
});
app.use(pinia);
const { default: i18n } = await import('./i18n');
app.use(i18n);
try {
const shell = useAppShellStore()
shell.setAppActive(true)
shell.setChatOwnsInput(true)
try {
window.__VUE_APP_ACTIVE__ = !!shell.appActive
} catch {
}
} catch {
}
app.use(router);
for (const [key, component] of Object.entries(ElementPlusIconsVue)) {
app.component(key, component);
}
app.mount('#app');
if (typeof performance !== 'undefined' && performance.mark) {
performance.mark('bootstrap_mount');
}
if (!readVanillaNoModUi()) {
void (async () => {
try {
await registerAllModRoutesFromGlob(router);
scheduleRouterAddressSync('glob');
} catch (e) {
console.warn('[bootstrap] mod routes (glob) after mount failed:', e);
}
})();
}
void (async () => {
if (readVanillaNoModUi()) return;
try {
const entries = await fetchModRoutesPayloadShared();
if (entries?.length) {
await registerModRoutes(router, entries);
scheduleRouterAddressSync('api');
```

```
}
} catch (e) {
console.warn('[bootstrap] Mod routes (API) register failed:', e);
}
})();
}
void bootstrap();
// 文件：api.test.js
import { describe, it, expect } from 'vitest'
describe('API Module Tests', () => {
it('should export all API modules', async () => {
const apiModules = await import('../src/api/index')
expect(apiModules).toBeDefined()
})
it('should have chat API', async () => {
const chatApi = await import('../src/api/chat')
expect(chatApi).toBeDefined()
})
it('should have products API', async () => {
const productsApi = await import('../src/api/products')
expect(productsApi).toBeDefined()
})
it('should have customers API', async () => {
const customersApi = await import('../src/api/customers')
expect(customersApi).toBeDefined()
})
it('should have orders API', async () => {
const ordersApi = await import('../src/api/orders')
expect(ordersApi).toBeDefined()
})
it('should have print API', async () => {
const printApi = await import('../src/api/print')
expect(printApi).toBeDefined()
})
it('should have wechat API', async () => {
const wechatApi = await import('../src/api/wechat')
expect(wechatApi).toBeDefined()
})
it('should have materials API', async () => {
const materialsApi = await import('../src/api/materials')
expect(materialsApi).toBeDefined()
})
})
describe('Utils Tests', () => {
it('should export utils', async () => {
const utils = await import('../src/utils/index')
expect(utils).toBeDefined()
})
})
```

```
describe('Composables Tests', () => {
it('should export useApi', async () => {
const useApi = await import('../src/composables/useApi')
expect(useApi).toBeDefined()
})
it('should export useProducts', async () => {
const useProducts = await import('../src/composables/useProducts')
expect(useProducts).toBeDefined()
})
})
// 文件：env.d.ts
declare module '*.vue' {
import type { DefineComponent } from 'vue';
const component: DefineComponent<Record<string, never>, Record<string, never>, unknown>;
export default component;
}
interface Window {
xcagiDesktop?: {
platform: NodeJS.Platform;
versions: Record<string, string>;
getDataDir: () => Promise<string>;
checkForUpdates: () => Promise<unknown>;
installUpdate: () => Promise<void>;
onUpdateEvent: (callback: (event: unknown) => void) => () => void;
getPairingQrPayload?: () => Promise<string>;
showNotification: (title: string, body: string) => Promise<void>;
setBadge: (count: number) => Promise<void>;
};
handleAutoAction?: (action: unknown, userMessage?: string) => void;
}
// 文件：tutorial/buildContext.ts
import type { VisibleNavItem } from '@/composables/useVisibleNavItems'
import type { ModForNavLabel } from '@/utils/coreNavLabel'
import type { TutorialBuildContext } from './types'
export function createTutorialBuildContext(options: {
industryId: string
mods: ModForNavLabel[]
visibleNav: VisibleNavItem[]
isProMode: boolean
}): TutorialBuildContext {
const modMenuKeys = new Set<string>()
for (const item of options.visibleNav) {
if (item.source === 'mod' && item.key) {
modMenuKeys.add(item.key)
}
}
return {
industryId: options.industryId,
```

```
mods: options.mods,
visibleNav: options.visibleNav,
isProMode: options.isProMode,
modMenuKeys,
}
}
// 文件：tutorial/runOnboardingTour.coverage.test.ts
import { describe, it, expect, vi, beforeEach, afterEach } from 'vitest'
const {
capturedDriverConfigRef,
mockHighlight,
mockDestroy,
mockWaitForSelector,
mockDemoGroupCleanup,
mockSleep,
mockGetVirtualCursor,
} = vi.hoisted(() => ({
capturedDriverConfigRef: { value: null as any },
mockHighlight: vi.fn(),
mockDestroy: vi.fn(),
mockWaitForSelector: vi.fn(),
mockDemoGroupCleanup: vi.fn(),
mockSleep: vi.fn().mockResolvedValue(undefined),
mockGetVirtualCursor: vi.fn(() => ({ hide: vi.fn() })),
}))
vi.mock('driver.js', () => ({
driver: vi.fn((config: any) => {
capturedDriverConfigRef.value = config
return {
highlight: mockHighlight,
destroy: mockDestroy,
}
}),
}))
vi.mock('driver.js/dist/driver.css', () => ({}))
vi.mock('@/tutorial/buildDriverSchedule', () => ({
demoGroupCleanup: mockDemoGroupCleanup,
waitForSelector: mockWaitForSelector,
}))
vi.mock('@/tutorial/demoHelpers', () => ({
getVirtualCursor: mockGetVirtualCursor,
makeTimerGroup: () => ({}),
sleep: mockSleep,
}))
import {
runOnboardingTour,
type RunOnboardingTourOptions,
type OnboardingTourStoreLike,
} from './runOnboardingTour'
```

```
function makeMockStore(overrides: Partial<OnboardingTourStoreLike> = {}): OnboardingTourStoreLike {
return {
paused: false,
skipRequested: false,
requestSkip: vi.fn(),
togglePause: vi.fn(),
...overrides,
}
}
function makeMockRouter(currentRouteName = 'home', currentQuery: Record<string, string> = {}) {
return {
currentRoute: { value: { name: currentRouteName, query: currentQuery } },
push: vi.fn().mockResolvedValue(undefined),
} as unknown as Parameters<typeof runOnboardingTour>[0]['router']
}
interface StepConfig {
id?: string
title?: string
description?: string
routeName?: string
routeQuery?: Record<string, string>
waitFor?: string
actionType?: 'click' | 'observe'
duration?: number
isLast?: boolean
demo?: () => { ok: boolean; fallbackMsg?: string }
}
function makeStep(overrides: StepConfig = {}): any {
return {
id: 'step-1',
title: 'Test',
description: 'Test step',
waitFor: '.test',
actionType: 'highlight' as const,
duration: 0,
isLast: true,
demo: () => ({ ok: true }),
...overrides,
}
}
function makeOptions(steps: any[], store?: OnboardingTourStoreLike): RunOnboardingTourOptions {
return {
steps,
router: makeMockRouter(),
store: store || makeMockStore(),
onComplete: vi.fn(),
onSkip: vi.fn(),
}
}
function setupPopoverWithFooter(footerClass = 'driver-popover-footer'): HTMLElement {
```

```
const popover = document.createElement('div')
popover.className = 'driver-popover'
const footer = document.createElement('div')
footer.className = footerClass
popover.appendChild(footer)
document.body.appendChild(popover)
return popover
}
function makeVisibleElement(tag = 'div'): HTMLElement {
const el = document.createElement(tag)
el.getBoundingClientRect = () => ({ width: 100, height: 50, top: 0, left: 0 }) as DOMRect
return el
}
beforeEach(() => {
vi.clearAllMocks()
capturedDriverConfigRef.value = null
mockWaitForSelector.mockResolvedValue(null)
mockSleep.mockResolvedValue(undefined)
document.body.innerHTML = ''
document.body.classList.remove('tutorial-active')
})
afterEach(() => {
vi.restoreAllMocks()
})
describe('runOnboardingTour - mountControls', () => {
it('通过 onPopoverRender 挂载控件（.driver-popover-footer）', () => {
const cleanup = runOnboardingTour(makeOptions([makeStep()]))
const popover = document.createElement('div')
const footer = document.createElement('div')
footer.className = 'driver-popover-footer'
popover.appendChild(footer)
capturedDriverConfigRef.value.onPopoverRender({ wrapper: popover })
const controls = footer.querySelector('.xcagi-tour-extra-controls')
expect(controls).not.toBeNull()
expect(controls?.querySelector('.xcagi-tour-btn-pause')).not.toBeNull()
expect(controls?.querySelector('.xcagi-tour-btn-skip')).not.toBeNull()
expect(controls?.querySelector('.xcagi-tour-btn-pause')?.textContent).toBe('暂停')
cleanup()
})
it('通过 .driver-popover-navigation-btns 查找 footer', () => {
const cleanup = runOnboardingTour(makeOptions([makeStep()]))
const popover = document.createElement('div')
const footer = document.createElement('div')
footer.className = 'driver-popover-navigation-btns'
popover.appendChild(footer)
capturedDriverConfigRef.value.onPopoverRender({ wrapper: popover })
expect(footer.querySelector('.xcagi-tour-extra-controls')).not.toBeNull()
cleanup()
})
it('footer 不存在时不挂载控件', () => {
```

```
const cleanup = runOnboardingTour(makeOptions([makeStep()]))
const popover = document.createElement('div')
capturedDriverConfigRef.value.onPopoverRender({ wrapper: popover })
expect(popover.querySelector('.xcagi-tour-extra-controls')).toBeNull()
cleanup()
})
it('已有控件时不重复挂载', () => {
const cleanup = runOnboardingTour(makeOptions([makeStep()]))
const popover = document.createElement('div')
const footer = document.createElement('div')
footer.className = 'driver-popover-footer'
const existing = document.createElement('div')
existing.className = 'xcagi-tour-extra-controls'
footer.appendChild(existing)
popover.appendChild(footer)
capturedDriverConfigRef.value.onPopoverRender({ wrapper: popover })
expect(footer.querySelectorAll('.xcagi-tour-extra-controls')).toHaveLength(1)
cleanup()
})
it('点击暂停按钮 → 调用 togglePause 并切换按钮文字', async () => {
const store = makeMockStore({ paused: false })
store.togglePause = vi.fn(() => {
store.paused = !store.paused
})
const cleanup = runOnboardingTour(makeOptions([makeStep()], store))
const popover = document.createElement('div')
const footer = document.createElement('div')
footer.className = 'driver-popover-footer'
popover.appendChild(footer)
capturedDriverConfigRef.value.onPopoverRender({ wrapper: popover })
const pauseBtn = footer.querySelector('.xcagi-tour-btn-pause') as HTMLButtonElement
expect(pauseBtn.textContent).toBe('暂停')
pauseBtn.click()
expect(store.togglePause).toHaveBeenCalled()
expect(pauseBtn.textContent).toBe('继续')
pauseBtn.click()
expect(pauseBtn.textContent).toBe('暂停')
cleanup()
})
it('store.paused=true 时按钮文字为"继续"', () => {
const store = makeMockStore({ paused: true })
const cleanup = runOnboardingTour(makeOptions([makeStep()], store))
const popover = document.createElement('div')
const footer = document.createElement('div')
footer.className = 'driver-popover-footer'
popover.appendChild(footer)
capturedDriverConfigRef.value.onPopoverRender({ wrapper: popover })
const pauseBtn = footer.querySelector('.xcagi-tour-btn-pause') as HTMLButtonElement
expect(pauseBtn.textContent).toBe('继续')
cleanup()
```

```
})
it('点击跳过按钮 → 调用 requestSkip', () => {
const store = makeMockStore()
const cleanup = runOnboardingTour(makeOptions([makeStep()], store))
const popover = document.createElement('div')
const footer = document.createElement('div')
footer.className = 'driver-popover-footer'
popover.appendChild(footer)
capturedDriverConfigRef.value.onPopoverRender({ wrapper: popover })
const skipBtn = footer.querySelector('.xcagi-tour-btn-skip') as HTMLButtonElement
skipBtn.click()
expect(store.requestSkip).toHaveBeenCalled()
cleanup()
})
})
describe('runOnboardingTour - navigateIfNeeded', () => {
it('routeName 为空时不导航', async () => {
const router = makeMockRouter('home')
const cleanup = runOnboardingTour({
steps: [makeStep({ routeName: undefined, isLast: true })],
router,
store: makeMockStore(),
onComplete: vi.fn(),
onSkip: vi.fn(),
})
await vi.waitFor(() => expect(mockWaitForSelector).toHaveBeenCalled())
expect(router.push).not.toHaveBeenCalled()
cleanup()
})
it('相同路由和 query 时不导航', async () => {
const router = makeMockRouter('chat', { tab: 'office' })
const cleanup = runOnboardingTour({
steps: [makeStep({ routeName: 'chat', routeQuery: { tab: 'office' }, isLast: true })],
router,
store: makeMockStore(),
onComplete: vi.fn(),
onSkip: vi.fn(),
})
await vi.waitFor(() => expect(mockWaitForSelector).toHaveBeenCalled())
expect(router.push).not.toHaveBeenCalled()
cleanup()
})
it('不同路由时执行导航', async () => {
const router = makeMockRouter('home')
const cleanup = runOnboardingTour({
steps: [makeStep({ routeName: 'chat', isLast: true })],
router,
store: makeMockStore(),
onComplete: vi.fn(),
onSkip: vi.fn(),
```

```
})
await vi.waitFor(() => expect(router.push).toHaveBeenCalledWith({ name: 'chat', query: {} }))
cleanup()
})
it('相同路由但不同 query 时执行导航', async () => {
const router = makeMockRouter('chat', { tab: 'old' })
const cleanup = runOnboardingTour({
steps: [makeStep({ routeName: 'chat', routeQuery: { tab: 'new' }, isLast: true })],
router,
store: makeMockStore(),
onComplete: vi.fn(),
onSkip: vi.fn(),
})
await vi.waitFor(() =>
expect(router.push).toHaveBeenCalledWith({ name: 'chat', query: { tab: 'new' } }),
)
cleanup()
})
it('router.push 抛错时不中断流程', async () => {
const router = makeMockRouter('home')
router.push = vi.fn().mockRejectedValue(new Error('nav error'))
const cleanup = runOnboardingTour({
steps: [makeStep({ routeName: 'chat', isLast: true })],
router,
store: makeMockStore(),
onComplete: vi.fn(),
onSkip: vi.fn(),
})
await vi.waitFor(() => expect(mockWaitForSelector).toHaveBeenCalled())
cleanup()
})
it('当前路由 name 为 undefined → 走 || 分支不报错', async () => {
const router = {
currentRoute: { value: { name: undefined, query: {} } },
push: vi.fn().mockResolvedValue(undefined),
} as unknown as Parameters<typeof runOnboardingTour>[0]['router']
const cleanup = runOnboardingTour({
steps: [makeStep({ routeName: 'chat', isLast: true })],
router,
store: makeMockStore(),
onComplete: vi.fn(),
onSkip: vi.fn(),
})
await vi.waitFor(() => expect(router.push).toHaveBeenCalledWith({ name: 'chat', query: {} }))
cleanup()
})
it('当前路由缺少 step 期望的 query key → 走 || 分支执行导航', async () => {
const router = {
currentRoute: { value: { name: 'chat', query: {} } },
push: vi.fn().mockResolvedValue(undefined),
```

```
} as unknown as Parameters<typeof runOnboardingTour>[0]['router']
const cleanup = runOnboardingTour({
steps: [makeStep({ routeName: 'chat', routeQuery: { tab: 'new' }, isLast: true })],
router,
store: makeMockStore(),
onComplete: vi.fn(),
onSkip: vi.fn(),
})
await vi.waitFor(() =>
expect(router.push).toHaveBeenCalledWith({ name: 'chat', query: { tab: 'new' } }),
)
cleanup()
})
})
describe('runOnboardingTour - runStep 元素查找', () => {
it('找到元素 → 调用 highlight', async () => {
const el = makeVisibleElement()
mockWaitForSelector.mockResolvedValue(el)
const cleanup = runOnboardingTour(makeOptions([makeStep({ isLast: true })]))
await vi.waitFor(() => expect(mockHighlight).toHaveBeenCalled())
expect(mockHighlight).toHaveBeenCalledWith(
expect.objectContaining({ element: el }),
)
cleanup()
})
it('未找到元素 + actionType=click → 重试等待', async () => {
mockWaitForSelector.mockResolvedValue(null)
const cleanup = runOnboardingTour(
makeOptions([makeStep({ actionType: 'click', isLast: true })]),
)
await vi.waitFor(() => expect(mockWaitForSelector).toHaveBeenCalledTimes(2))
expect(mockHighlight).not.toHaveBeenCalled()
cleanup()
})
it('未找到元素 + actionType=observe → 不重试', async () => {
mockWaitForSelector.mockResolvedValue(null)
const cleanup = runOnboardingTour(
makeOptions([makeStep({ actionType: 'observe', isLast: true })]),
)
await vi.waitFor(() => expect(mockWaitForSelector).toHaveBeenCalled())
await new Promise((r) => setTimeout(r, 100))
expect(mockWaitForSelector).toHaveBeenCalledTimes(1)
cleanup()
})
it('元素在 sidebar 中 → 额外 sleep + refresh highlight', async () => {
const sidebar = document.createElement('div')
sidebar.className = 'sidebar'
const el = makeVisibleElement()
sidebar.appendChild(el)
document.body.appendChild(sidebar)
```

```
mockWaitForSelector.mockResolvedValue(el)
const cleanup = runOnboardingTour(makeOptions([makeStep({ isLast: true })]))
await vi.waitFor(() => expect(mockHighlight).toHaveBeenCalledTimes(2))
cleanup()
})
})
describe('runOnboardingTour - demo 回退消息', () => {
it('demo 失败且有 fallbackMsg → 用 fallback 重新 highlight', async () => {
const el = makeVisibleElement()
mockWaitForSelector.mockResolvedValue(el)
const cleanup = runOnboardingTour(
makeOptions([
makeStep({
isLast: true,
demo: () => ({ ok: false, fallbackMsg: '回退提示文案' }),
}),
]),
)
await vi.waitFor(() => {
const calls = mockHighlight.mock.calls
const fallbackCall = calls.find(
(call: any[]) => call[0]?.popover?.description === '回退提示文案',
)
expect(fallbackCall).toBeDefined()
})
cleanup()
})
it('demo 成功 → 不触发 fallback highlight', async () => {
const el = makeVisibleElement()
mockWaitForSelector.mockResolvedValue(el)
const cleanup = runOnboardingTour(
makeOptions([
makeStep({
isLast: true,
demo: () => ({ ok: true }),
}),
]),
)
await vi.waitFor(() => expect(mockHighlight).toHaveBeenCalled())
const calls = mockHighlight.mock.calls
const hasFallback = calls.some(
(call: any[]) => call[0]?.popover?.description === '回退提示文案',
)
expect(hasFallback).toBe(false)
cleanup()
})
})
describe('runOnboardingTour - isLast 步骤', () => {
it('isLast 步骤 → 添加完成按钮', async () => {
const el = makeVisibleElement()
```

```
mockWaitForSelector.mockResolvedValue(el)
setupPopoverWithFooter('driver-popover-footer')
const onComplete = vi.fn()
const cleanup = runOnboardingTour({
steps: [makeStep({ isLast: true })],
router: makeMockRouter(),
store: makeMockStore(),
onComplete,
onSkip: vi.fn(),
})
await vi.waitFor(() => {
const cta = document.querySelector('.xcagi-tour-extra-controls')
expect(cta?.querySelector('.xcagi-tour-btn-finish')).not.toBeNull()
})
cleanup()
})
it('点击完成按钮 → 调用 onComplete', async () => {
const el = makeVisibleElement()
mockWaitForSelector.mockResolvedValue(el)
setupPopoverWithFooter('driver-popover-footer')
const onComplete = vi.fn()
const cleanup = runOnboardingTour({
steps: [makeStep({ isLast: true })],
router: makeMockRouter(),
store: makeMockStore(),
onComplete,
onSkip: vi.fn(),
})
await vi.waitFor(() => {
expect(document.querySelector('.xcagi-tour-btn-finish')).not.toBeNull()
})
const finishBtn = document.querySelector('.xcagi-tour-btn-finish') as HTMLButtonElement
finishBtn.click()
expect(onComplete).toHaveBeenCalled()
})
})
describe('runOnboardingTour - 多步骤流程', () => {
it('两步骤流程 → 第一步完成后进入第二步', async () => {
const el = makeVisibleElement()
mockWaitForSelector.mockResolvedValue(el)
setupPopoverWithFooter('driver-popover-footer')
const onComplete = vi.fn()
const cleanup = runOnboardingTour({
steps: [
makeStep({
id: 'step-1',
isLast: false,
duration: 0,
}),
makeStep({
```

```
id: 'step-2',
isLast: true,
}),
],
router: makeMockRouter(),
store: makeMockStore(),
onComplete,
onSkip: vi.fn(),
})
await vi.waitFor(
() => {
expect(document.querySelector('.xcagi-tour-btn-finish')).not.toBeNull()
},
{ timeout: 3000 },
)
cleanup()
})
it('所有步骤完成 → 调用 onComplete', async () => {
const el = makeVisibleElement()
mockWaitForSelector.mockResolvedValue(el)
const onComplete = vi.fn()
const cleanup = runOnboardingTour({
steps: [
makeStep({
id: 'step-1',
isLast: false,
duration: 0,
}),
],
router: makeMockRouter(),
store: makeMockStore(),
onComplete,
onSkip: vi.fn(),
})
await vi.waitFor(() => expect(onComplete).toHaveBeenCalled(), { timeout: 3000 })
cleanup()
})
})
describe('runOnboardingTour - skip 与 destroy', () => {
it('skipRequested 在步骤开始时为 true → 调用 onSkip', async () => {
const store = makeMockStore({ skipRequested: true })
const onSkip = vi.fn()
const cleanup = runOnboardingTour({
steps: [makeStep({ isLast: false })],
router: makeMockRouter(),
store,
onComplete: vi.fn(),
onSkip,
})
await vi.waitFor(() => expect(onSkip).toHaveBeenCalled(), { timeout: 2000 })
```

```
cleanup()
})
it('onDestroyStarted → 触发 cleanup', async () => {
const el = makeVisibleElement()
mockWaitForSelector.mockResolvedValue(el)
const cleanup = runOnboardingTour(makeOptions([makeStep({ isLast: true })]))
await vi.waitFor(() => expect(mockHighlight).toHaveBeenCalled())
capturedDriverConfigRef.value.onDestroyStarted()
expect(document.body.classList.contains('tutorial-active')).toBe(false)
})
})
describe('runOnboardingTour - body class', () => {
it('运行时添加 tutorial-active class', () => {
const cleanup = runOnboardingTour(makeOptions([makeStep()]))
expect(document.body.classList.contains('tutorial-active')).toBe(true)
cleanup()
})
it('cleanup 后移除 tutorial-active class', () => {
const cleanup = runOnboardingTour(makeOptions([makeStep()]))
cleanup()
expect(document.body.classList.contains('tutorial-active')).toBe(false)
})
})
describe('runOnboardingTour - 空步骤', () => {
it('空步骤 → 立即调用 onSkip', () => {
const onSkip = vi.fn()
const onComplete = vi.fn()
const cleanup = runOnboardingTour({
steps: [],
router: makeMockRouter(),
store: makeMockStore(),
onComplete,
onSkip,
})
expect(onSkip).toHaveBeenCalled()
expect(onComplete).not.toHaveBeenCalled()
cleanup()
})
})
describe('runOnboardingTour - cleanup 幂等性', () => {
it('多次调用 cleanup 不抛错', () => {
const cleanup = runOnboardingTour(makeOptions([makeStep()]))
cleanup()
expect(() => cleanup()).not.toThrow()
})
it('cleanup 时 driver.destroy() 抛错 → catch 分支不中断', () => {
mockDestroy.mockImplementation(() => {
throw new Error('destroy error')
})
const cleanup = runOnboardingTour(makeOptions([makeStep()]))
```

```
expect(() => cleanup()).not.toThrow()
})
})
describe('runOnboardingTour - waitWithPause 分支', () => {
it('paused 状态下等待 → 走 paused 分支并最终 skip', async () => {
let currentTime = 0
const dateSpy = vi.spyOn(Date, 'now').mockImplementation(() => currentTime)
const store = makeMockStore({ paused: true })
const onSkip = vi.fn()
let sleepCallCount = 0
mockSleep.mockImplementation(async (ms: number) => {
sleepCallCount++
currentTime += ms
if (sleepCallCount >= 2) {
store.skipRequested = true
}
return undefined
})
const cleanup = runOnboardingTour({
steps: [makeStep({ isLast: false, duration: 500 })],
router: makeMockRouter(),
store,
onComplete: vi.fn(),
onSkip,
})
await vi.waitFor(() => expect(onSkip).toHaveBeenCalled(), { timeout: 3000 })
dateSpy.mockRestore()
cleanup()
})
it('正常等待完成 → 走 normal sleep 分支并进入下一步', async () => {
let currentTime = 0
const dateSpy = vi.spyOn(Date, 'now').mockImplementation(() => currentTime)
const onComplete = vi.fn()
mockSleep.mockImplementation(async (ms: number) => {
currentTime += ms
return undefined
})
const cleanup = runOnboardingTour({
steps: [makeStep({ isLast: false, duration: 500 })],
router: makeMockRouter(),
store: makeMockStore(),
onComplete,
onSkip: vi.fn(),
})
await vi.waitFor(() => expect(onComplete).toHaveBeenCalled(), { timeout: 3000 })
dateSpy.mockRestore()
cleanup()
})
})
```

```
// 文件：tutorial/virtualCursor.types.ts
export interface VirtualCursorMoveOptions {
duration?: number
click?: boolean
label?: string
}
export interface VirtualCursorClickOptions {
duration?: number
label?: string
}
export interface VirtualCursorApi {
moveTo(
target: HTMLElement | { x: number; y: number },
options?: VirtualCursorMoveOptions,
): void
click(target: HTMLElement, options?: VirtualCursorClickOptions): void
hide(): void
show(): void
}
declare global {
interface Window {
virtualCursor?: VirtualCursorApi
}
}
export {}
// 文件：tutorial/buildQuickStartOfficeImportTour.test.ts
import { describe, expect, it } from 'vitest'
import { buildQuickStartOfficeImportSteps } from './buildQuickStartOfficeImportTour'
describe('buildQuickStartOfficeImportSteps', () => {
it('covers excel x2, word, and cleanup after office pack', () => {
const steps = buildQuickStartOfficeImportSteps()
const ids = steps.map((s) => s.id)
expect(ids[0]).toBe('quickstart-import-go-chat')
expect(ids).toContain('quickstart-import-excel-a')
expect(ids).toContain('quickstart-import-excel-b')
expect(ids).toContain('quickstart-import-word')
expect(ids[ids.length - 1]).toBe('quickstart-import-word-result')
expect(steps.every((s) => s.routeName === 'chat')).toBe(true)
})
})
// 文件：tutorial/promptAdvancedTutorial.ts
import { nextTick } from 'vue'
import type { Router } from 'vue-router'
import { appConfirm } from '@/utils/appDialog'
import { useOnboardingTutorialStore } from '@/stores/onboardingTutorial'
import type { OnboardingReturnContext } from '@/stores/onboardingTutorial'
import type { TutorialBuildContext } from '@/tutorial/types'
const DEFAULT_INSTALL_MESSAGE =
```

```
'安装已完成，可以开始使用了。\n\n是否现在观看进阶教程，快速熟悉菜单与智能对话？'
export async function launchAdvancedDriverTour(options: {
router: Router
buildContext: TutorialBuildContext
returnContext?: OnboardingReturnContext
skipNavigation?: boolean
}): Promise<boolean> {
const store = useOnboardingTutorialStore()
const { router, buildContext, returnContext, skipNavigation = false } = options
if (!skipNavigation) {
await router.push({ name: 'chat' }).catch(() => {})
await nextTick()
for (let i = 0; i < 4; i += 1) {
const newConversationBtn = document.getElementById('newConversationBtn')
if (newConversationBtn) {
newConversationBtn.dispatchEvent(new MouseEvent('click', { bubbles: true, cancelable: true }))
break
}
await new Promise((resolve) => window.setTimeout(resolve, 80))
}
}
store.start({
track: 'advanced',
buildContext,
returnContext: returnContext ?? { routeName: 'chat' },
})
return store.active
}
export type InstallTutorialPromptResult = 'started' | 'dismissed' | 'already_completed'
export async function promptAdvancedTutorialAfterInstall(options: {
router: Router
buildContext: TutorialBuildContext
message?: string
returnContext?: OnboardingReturnContext
skipIfCompleted?: boolean
}): Promise<InstallTutorialPromptResult> {
const {
router,
buildContext,
message = DEFAULT_INSTALL_MESSAGE,
returnContext,
skipIfCompleted = true,
} = options
const store = useOnboardingTutorialStore()
if (skipIfCompleted && store.isCompleted()) {
return 'already_completed'
}
const watch = await appConfirm(message, {
title: '安装完成',
confirmText: '观看教程',
```

```
cancelText: '稍后再说',
})
if (!watch) return 'dismissed'
const started = await launchAdvancedDriverTour({ router, buildContext, returnContext })
return started ? 'started' : 'dismissed'
}
export function resolveRouteNameFromPath(router: Router, path: string): string {
const raw = String(path || '').trim()
if (!raw) return 'chat'
try {
const resolved = router.resolve(raw)
return String(resolved.name || 'chat')
} catch {
return 'chat'
}
}
// 文件：tutorial/tutorialDbSampleDemo.ts
import { customersApi } from '@/api/customers'
import { productsApi } from '@/api/products'
import { TUTORIAL_SAMPLE_NAME_PREFIX } from '@/constants/tutorialSamples'
import { sleep } from './demoHelpers'
const PREFIX = TUTORIAL_SAMPLE_NAME_PREFIX
const TUTORIAL_UNIT = `${PREFIX}演示单位`
const TUTORIAL_DB_IDS_KEY = 'xcagi_tutorial_db_sample_ids'
export type TutorialDbSampleIds = {
customerIds: number[]
productIds: number[]
}
function readStoredIds(): TutorialDbSampleIds {
if (typeof window === 'undefined') return { customerIds: [], productIds: [] }
try {
const raw = sessionStorage.getItem(TUTORIAL_DB_IDS_KEY)
if (!raw) return { customerIds: [], productIds: [] }
const parsed = JSON.parse(raw) as TutorialDbSampleIds
return {
customerIds: (parsed.customerIds || []).map(Number).filter(Boolean),
productIds: (parsed.productIds || []).map(Number).filter(Boolean),
}
} catch {
return { customerIds: [], productIds: [] }
}
}
function storeIds(ids: TutorialDbSampleIds): void {
if (typeof window === 'undefined') return
try {
sessionStorage.setItem(TUTORIAL_DB_IDS_KEY, JSON.stringify(ids))
} catch {
}
}
```

```
export function clearTutorialDbSampleIds(): void {
if (typeof window === 'undefined') return
try {
sessionStorage.removeItem(TUTORIAL_DB_IDS_KEY)
} catch {
}
}
export async function seedQuickStartTutorialDbSamples(): Promise<TutorialDbSampleIds> {
const customerIds: number[] = []
const productIds: number[] = []
for (const name of [`${PREFIX}市场部`, `${PREFIX}研发部`]) {
try {
const r = await customersApi.createCustomer({
name,
contact_person: '教程样本',
})
const id = Number(r?.data?.id)
if (id) customerIds.push(id)
} catch {
}
}
const products = [
{ model_number: 'TUT-A1', name: `${PREFIX}签字笔` },
{ model_number: 'TUT-A2', name: `${PREFIX}笔记本` },
{ model_number: 'TUT-B1', name: `${PREFIX}张三` },
{ model_number: 'TUT-B2', name: `${PREFIX}李四` },
]
for (const p of products) {
try {
const r = await productsApi.createProduct({
...p,
unit: TUTORIAL_UNIT,
price: 1,
quantity: 1,
})
const id = Number(r?.data?.id)
if (id) productIds.push(id)
} catch {
}
}
const ids = { customerIds, productIds }
storeIds(ids)
return ids
}
export async function purgeQuickStartTutorialDbSamples(): Promise<void> {
const stored = readStoredIds()
if (stored.customerIds.length) {
try {
await customersApi.batchDeleteCustomers(stored.customerIds)
} catch {
```

```
}
}
if (stored.productIds.length) {
try {
await productsApi.batchDeleteProducts(stored.productIds)
} catch {
}
}
clearTutorialDbSampleIds()
}
function selectRowsWithPrefix(containerSelector: string, prefix: string): number {
const root = document.querySelector(containerSelector)
if (!root) return 0
let count = 0
root.querySelectorAll('tbody tr').forEach((tr) => {
const text = (tr.textContent || '').trim()
if (!text.includes(prefix)) return
const cb = tr.querySelector<HTMLInputElement>('input[type="checkbox"]')
if (cb && !cb.checked) {
cb.click()
count += 1
}
})
return count
}
async function confirmBatchDeleteDialog(): Promise<void> {
await sleep(350)
const btn = document.querySelector<HTMLElement>(
'.confirm-dialog-footer .btn-danger, .confirm-dialog .btn-danger',
)
btn?.click()
}
export async function runQuickStartDeleteCustomersDemo(): Promise<void> {
await sleep(500)
selectRowsWithPrefix('#view-customers', PREFIX)
await sleep(300)
const batchBtn = document.querySelector<HTMLElement>(
'#view-customers .customers-header-actions .btn-danger, #view-customers .btn-danger',
)
if (batchBtn) {
batchBtn.click()
await confirmBatchDeleteDialog()
}
const stored = readStoredIds()
if (stored.customerIds.length) {
try {
await customersApi.batchDeleteCustomers(stored.customerIds)
} catch {
}
}
```

```
storeIds({ ...stored, customerIds: [] })
}
export async function runQuickStartDeleteProductsDemo(): Promise<void> {
await sleep(500)
const unitSelect = document.querySelector<HTMLSelectElement>('#view-products .search-box select')
if (unitSelect) {
const hasUnit = Array.from(unitSelect.options).some((o) => o.value === TUTORIAL_UNIT)
if (!hasUnit) {
const opt = document.createElement('option')
opt.value = TUTORIAL_UNIT
opt.textContent = TUTORIAL_UNIT
unitSelect.appendChild(opt)
}
unitSelect.value = TUTORIAL_UNIT
unitSelect.dispatchEvent(new Event('change', { bubbles: true }))
await sleep(800)
}
selectRowsWithPrefix('#view-products', PREFIX)
await sleep(300)
const batchBtn = document.querySelector<HTMLElement>('#view-products .page-header .btn-danger')
if (batchBtn) {
batchBtn.click()
await confirmBatchDeleteDialog()
}
const stored = readStoredIds()
if (stored.productIds.length) {
try {
await productsApi.batchDeleteProducts(stored.productIds)
} catch {
}
}
clearTutorialDbSampleIds()
}
// 文件：tutorial/demoHelpers.ts
import type { VirtualCursorApi } from './virtualCursor.types'
export const TUTORIAL_DEMO_SPEED = 2.0
export type DemoResult = { ok: boolean; fallbackMsg?: string }
export type TimerGroup = {
set: (fn: () => void, ms: number) => void
clear: () => void
}
export function makeTimerGroup(): TimerGroup {
const ids: number[] = []
return {
set(fn: () => void, ms: number) {
ids.push(window.setTimeout(fn, ms))
},
clear() {
ids.forEach((id) => window.clearTimeout(id))
```

```
ids.length = 0
},
}
}
export function getVirtualCursor(): VirtualCursorApi | undefined {
return window.virtualCursor
}
export function cursorClick(
el: HTMLElement,
label?: string,
duration = 700 * TUTORIAL_DEMO_SPEED,
): void {
const vc = getVirtualCursor()
if (vc) {
vc.click(el, { duration, label })
}
window.setTimeout(() => {
try {
el.scrollIntoView({ behavior: 'smooth', block: 'center' })
} catch {
}
el.click()
}, duration)
}
export function safeClick(
selector: string,
label?: string,
duration = 700 * TUTORIAL_DEMO_SPEED,
): boolean {
const el = document.querySelector<HTMLElement>(selector)
if (!el) return false
try {
el.scrollIntoView({ behavior: 'smooth', block: 'center' })
} catch {
}
cursorClick(el, label, duration)
return true
}
export function fireKey(key: string, code: string, meta = false): void {
const isMac =
typeof navigator !== 'undefined' &&
/Mac|iPhone|iPad/i.test(navigator.platform || navigator.userAgent)
const ev = new KeyboardEvent('keydown', {
key,
code,
metaKey: meta && isMac,
ctrlKey: meta && !isMac,
bubbles: true,
})
document.dispatchEvent(ev)
```

```
}
export function highlightElement(el: HTMLElement, ms = 1500 * TUTORIAL_DEMO_SPEED): void {
el.style.transition = 'outline .3s, outline-offset .3s'
el.style.outline = '2px solid #3b82f6'
el.style.outlineOffset = '4px'
window.setTimeout(() => {
el.style.outline = ''
el.style.outlineOffset = ''
}, ms)
}
export function sleep(ms: number): Promise<void> {
return new Promise((resolve) => window.setTimeout(resolve, ms))
}
// 文件：tutorial/buildQuickStartDeleteTutorial.test.ts
import { describe, expect, it } from 'vitest'
import { buildQuickStartDeleteTutorialSteps } from './buildQuickStartDeleteTutorial'
import type { VisibleNavItem } from '@/composables/useVisibleNavItems'
const nav = (keys: Array<{ key: string; name: string }>): VisibleNavItem[] =>
keys.map((k) => ({
key: k.key,
name: k.name,
routeName: k.key,
source: 'core',
}))
describe('buildQuickStartDeleteTutorialSteps', () => {
it('returns empty when customers and products are absent', () => {
expect(buildQuickStartDeleteTutorialSteps(nav([{ key: 'chat', name: '对话' }]))).toEqual([])
})
it('includes department and personnel delete steps when nav present', () => {
const steps = buildQuickStartDeleteTutorialSteps(
nav([
{ key: 'products', name: '人员管理' },
{ key: 'customers', name: '部门管理' },
]),
)
const ids = steps.map((s) => s.id)
expect(ids).toContain('quickstart-import-seed-db')
expect(ids).toContain('quickstart-delete-customers-nav')
expect(ids).toContain('quickstart-delete-customers')
expect(ids).toContain('quickstart-delete-products-nav')
expect(ids).toContain('quickstart-delete-products')
expect(ids[ids.length - 1]).toBe('quickstart-import-cleanup')
})
})
// 文件：tutorial/zeroCoverage.test.ts
import { describe, expect, it, beforeEach, vi, afterEach } from 'vitest'
import { QUICK_START_PAGE_HIGHLIGHTS } from './quickStartPageHighlights'
import {
```

```
QUICK_START_FOCUS_NAV_KEYS,
QUICK_START_PAGE_ROUTE,
QUICK_START_NAV_INTRO,
} from './quickStartNav'
import { buildAssistantFloatSteps } from './buildAssistantFloatTour'
import { closeAssistantFloatPanelForTutorial, isAssistantFloatPanelOpen } from './assistantFloatTutorial'
import { isOnboardingDriverTutorialActive } from './onboardingTutorialActive'
import {
TUTORIAL_DEMO_SPEED,
makeTimerGroup,
getVirtualCursor,
cursorClick,
safeClick,
fireKey,
highlightElement,
sleep,
} from './demoHelpers'
import {
buildDriverScheduleFromTutorialSteps,
waitForSelector,
demoGroupCleanup,
} from './buildDriverSchedule'
import { filterStepsForPro, resolveTrackSteps, resolveAllWarmupSteps } from './resolveSteps'
import {
fetchTutorialSampleFile,
assignFileToInput,
injectExcelAnalyzeSample,
uploadOfficeSampleForPath,
readWordSampleViaOfficePack,
runQuickStartExcelDemo,
runQuickStartWordDemo,
waitForChatContains,
cleanupQuickStartImportDemo,
} from './tutorialOfficeImportDemo'
import {
clearTutorialDbSampleIds,
seedQuickStartTutorialDbSamples,
purgeQuickStartTutorialDbSamples,
runQuickStartDeleteCustomersDemo,
runQuickStartDeleteProductsDemo,
} from './tutorialDbSampleDemo'
import {
launchAdvancedDriverTour,
promptAdvancedTutorialAfterInstall,
resolveRouteNameFromPath,
} from './promptAdvancedTutorial'
import type { TutorialStep } from './types'
class MockDataTransfer {
files: File[] = []
items = {
```

```
add: (file: File) => {
this.files.push(file)
},
}
}
;(globalThis as unknown as { DataTransfer: unknown }).DataTransfer = MockDataTransfer
function patchInputFilesSetter(input: HTMLInputElement): void {
Object.defineProperty(input, 'files', {
configurable: true,
get() {
return (this as unknown as { _mockFiles?: File[] })._mockFiles || null
},
set(value: File[]) {
;(this as unknown as { _mockFiles?: File[] })._mockFiles = value
},
})
}
vi.mock('@/api/customers', () => ({
customersApi: {
createCustomer: vi.fn(),
batchDeleteCustomers: vi.fn(),
},
}))
vi.mock('@/api/products', () => ({
productsApi: {
createProduct: vi.fn(),
batchDeleteProducts: vi.fn(),
},
}))
vi.mock('@/utils/officeEmployeeReadApi', () => ({
uploadTutorialOfficeFile: vi.fn(),
readWordViaOfficePack: vi.fn(),
}))
vi.mock('@/utils/platformShellApi', () => ({
fetchEmployeePlannerStatus: vi.fn(),
}))
vi.mock('@/utils/appDialog', () => ({
appConfirm: vi.fn(),
}))
const officeApiMock = await import('@/utils/officeEmployeeReadApi')
const customersApiMock = (await import('@/api/customers')).customersApi
const productsApiMock = (await import('@/api/products')).productsApi
const appConfirmMock = (await import('@/utils/appDialog')).appConfirm
describe('quickStartPageHighlights', () => {
it('exports highlights for chat, workflow-employee-space, im, and settings', () => {
expect(Object.keys(QUICK_START_PAGE_HIGHLIGHTS).sort()).toEqual(
['chat', 'im', 'settings', 'workflow-employee-space'].sort(),
)
})
it('chat highlights have three entries with targetSelector', () => {
```

```
const chatHighlights = QUICK_START_PAGE_HIGHLIGHTS.chat
expect(chatHighlights).toHaveLength(3)
for (const h of chatHighlights) {
expect(typeof h.idSuffix).toBe('string')
expect(typeof h.title).toBe('string')
expect(typeof h.description).toBe('string')
expect(typeof h.targetSelector).toBe('string')
}
})
it('workflow-employee-space highlights have head and desks entries', () => {
const highlights = QUICK_START_PAGE_HIGHLIGHTS['workflow-employee-space']
expect(highlights).toHaveLength(2)
expect(highlights[0].idSuffix).toBe('head')
expect(highlights[1].idSuffix).toBe('desks')
})
it('settings highlights include model-payment and recharge entries', () => {
const highlights = QUICK_START_PAGE_HIGHLIGHTS.settings
expect(highlights.length).toBeGreaterThanOrEqual(3)
const suffixes = highlights.map((h) => h.idSuffix)
expect(suffixes).toContain('model-payment')
expect(suffixes).toContain('recharge')
})
it('im highlights have sidebar, thread, compose entries', () => {
const highlights = QUICK_START_PAGE_HIGHLIGHTS.im
expect(highlights).toHaveLength(3)
const suffixes = highlights.map((h) => h.idSuffix)
expect(suffixes).toEqual(['sidebar', 'thread', 'compose'])
})
})
describe('quickStartNav', () => {
it('QUICK_START_FOCUS_NAV_KEYS has five entries in order', () => {
expect(QUICK_START_FOCUS_NAV_KEYS).toEqual([
'chat',
'ai-ecosystem',
'employee-workflow',
'im',
'settings',
])
})
it('QUICK_START_PAGE_ROUTE maps employee-workflow to workflow-employee-space', () => {
expect(QUICK_START_PAGE_ROUTE['employee-workflow']).toBe('workflow-employee-space')
})
it('QUICK_START_PAGE_ROUTE only has employee-workflow mapping', () => {
expect(Object.keys(QUICK_START_PAGE_ROUTE)).toEqual(['employee-workflow'])
})
it('QUICK_START_NAV_INTRO has intro text for all five nav keys', () => {
for (const key of QUICK_START_FOCUS_NAV_KEYS) {
expect(QUICK_START_NAV_INTRO[key]).toBeTruthy()
expect(typeof QUICK_START_NAV_INTRO[key]).toBe('string')
expect(QUICK_START_NAV_INTRO[key]!.length).toBeGreaterThan(0)
```

```
snippet = str(item.get("snippet") or "")[:600]
if url:
out.append({"title": title, "url": url, "snippet": snippet})
return out
def _format_kitten_business_snapshot_block(self, snap: Any | None) -> str:
if snap is None or (isinstance(snap, dict) and not snap):
return ""
if not isinstance(snap, dict):
return ""
preview = snap.get("text")
if not isinstance(preview, str) or not preview.strip():
return ""
head = "【小猫分析· 业务数据库快照】"
ga = snap.get("generated_at")
if ga:
head += f"（生成时间 {ga}）"
return f"{head}\n{preview.strip()}"
def _format_kitten_dataset_block(self, kd: Any | None) -> str:
if kd is None or (isinstance(kd, dict) and not kd):
return "【小猫分析】当前未附带表格数据，请根据通用数据分析知识回答用户问题。"
if not isinstance(kd, dict):
return ""
lines = ["【小猫分析· 数据上下文】"]
fn = kd.get("file_name") or kd.get("name")
if fn:
lines.append(f"文件名：{fn}")
if kd.get("rows") is not None:
lines.append(f"行数：{kd.get('rows')}")
if kd.get("columns") is not None:
lines.append(f"列数：{kd.get('columns')}")
fields = kd.get("fields") or kd.get("field_names")
if isinstance(fields, (list, tuple)) and fields:
lines.append(f"字段：{', '.join(str(x) for x in fields[:80])}")
if len(fields) > 80:
lines.append("…（字段列表已省略）")
preview = kd.get("preview_text")
if isinstance(preview, str) and preview.strip():
lines.append("样本行（供理解表格结构）：")
lines.append(preview.strip())
lines.append("（若字段名为 __EMPTY 等占位，请结合样本行推断含义。）")
return "\n".join(lines)
def _format_web_search_block(
self,
hits: Any,
err: Any,
meta: Any,
) -> str:
lines: list[str] = ["【互联网检索摘要】"]
if isinstance(meta, dict):
prov = str(meta.get("provider") or "").strip()
```

```
q = str(meta.get("query") or "").strip()
if prov or q:
lines.append(f"提供方：{prov or '-'}；检索查询：{q or '-'}")
safe_hits = hits if isinstance(hits, list) else []
if not safe_hits:
if err:
lines.append(f"本次未拿到网页摘要：{str(err)[:400]}")
lines.append("请结合用户上传数据、业务库快照（若有）与自身知识回答；勿虚构检索结果。")
return "\n".join(lines)
lines.append("以下条目供引用（请在回答中标注来源序号或链接）：")
for idx, item in enumerate(safe_hits[:8], start=1):
if not isinstance(item, dict):
continue
title = str(item.get("title") or "").strip()
url = str(item.get("url") or "").strip()
snip = str(item.get("snippet") or "").strip()
if len(snip) > 500:
snip = snip[:500] + "…"
lines.append(f"{idx}. {title or url}\n {url}\n {snip}")
lines.append("严禁编造以上列表中不存在的链接或摘要；信息不足时请明确说明。")
return "\n".join(lines)
def _format_request_context_for_system(self, req: dict[str, Any] | None) -> str:
if not req or not isinstance(req, dict):
return ""
blocks: list[str] = []
if req.get("kitten_analyzer"):
blocks.append(self._format_kitten_dataset_block(req.get("kitten_dataset")))
db_block = self._format_kitten_business_snapshot_block(
req.get("kitten_business_snapshot")
)
if db_block:
blocks.append(db_block)
if req.get("kitten_web_search"):
blocks.append(
self._format_web_search_block(
req.get("web_search_results"),
req.get("web_search_error"),
req.get("web_search_meta"),
)
)
excel_vector_ctx = req.get("excel_vector_context")
if isinstance(excel_vector_ctx, dict):
blocks.append(self._format_excel_vector_block(excel_vector_ctx))
extra = {
k: v
for k, v in req.items()
if k
not in (
"kitten_analyzer",
"has_dataset",
```

```
"kitten_dataset",
"kitten_include_business_db",
"kitten_business_snapshot",
"kitten_web_search",
"web_search_results",
"web_search_error",
"web_search_meta",
"excel_vector_context",
)
}
if extra:
try:
dumped = json.dumps(extra, ensure_ascii=False, default=str)
if len(dumped) > 4096:
dumped = dumped[:4096] + "…"
blocks.append(f"【附加上下文】\n{dumped}")
except RECOVERABLE_ERRORS:
logger.debug("suppressed exception", exc_info=True)
merged = "\n\n".join(b for b in blocks if b)
return merged
def _format_excel_vector_block(self, payload: dict[str, Any]) -> str:
index_id = str(payload.get("index_id") or "").strip()
query = str(payload.get("query") or "").strip()
hits = payload.get("hits") if isinstance(payload.get("hits"), list) else []
lines = ["【Excel语义检索上下文】"]
if index_id:
lines.append(f"索引ID：{index_id}")
if query:
lines.append(f"问题：{query}")
if not hits:
lines.append("未召回相关内容。")
lines.append("若信息不足，请明确告知用户并引导其补充筛选条件。")
return "\n".join(lines)
lines.append("以下是最相关的表格片段（按相关度排序）：")
for idx, hit in enumerate(hits[:8], start=1):
score = float(hit.get("score", 0.0))
metadata = hit.get("metadata") if isinstance(hit.get("metadata"), dict) else {}
sheet = str(metadata.get("sheet") or "-")
row_index = metadata.get("row_index")
content = str(hit.get("content") or "").strip()
if len(content) > 600:
content = content[:600] + "..."
if row_index is not None:
lines.append(f"{idx}. sheet={sheet}, row={row_index}, score={score:.4f}")
else:
lines.append(f"{idx}. sheet={sheet}, score={score:.4f}")
lines.append(content)
lines.append("回答时优先引用以上片段，不要编造未出现的数据。")
return "\n".join(lines)
def _metadata_cache_hash(self, metadata: dict[str, Any] | None) -> str:
```

```
if not metadata:
return ""
try:
return hashlib.md5(
json.dumps(metadata, sort_keys=True, ensure_ascii=False, default=str).encode(
"utf-8"
)
).hexdigest()
except RECOVERABLE_ERRORS:
return str(hash(frozenset(metadata.keys())))
def _build_context_prompt(self, context) -> str:
blocks: list[str] = []
req = (context.metadata or {}).get("request_context")
req_block = self._format_request_context_for_system(req)
if req_block:
blocks.append(req_block)
session_parts: list[str] = []
if context.current_intent:
session_parts.append(f"当前会话意图：{context.current_intent}")
if context.current_tool_key:
session_parts.append(f"当前工具：{context.current_tool_key}")
if context.intent_hints:
session_parts.append(f"意图线索：{', '.join(context.intent_hints)}")
if context.pending_confirmation:
action = context.pending_confirmation.get("action", "")
desc = context.pending_confirmation.get("description", "")
session_parts.append(f"待确认操作：{action} - {desc}")
if context.last_action:
session_parts.append(f"最近操作：{context.last_action}")
if session_parts:
blocks.append("【当前会话上下文】\n" + "\n".join(session_parts) + "\n【END会话上下文】")
if not blocks:
return ""
return "\n\n".join(blocks)
// 文件：services/conversation/api.py
import logging
import time
from typing import Any, cast
from app.neuro_bus.event_publisher_mixin import NeuroEventPublisherMixin
from app.services.conversation.context import ConversationContext
from app.utils.cache_manager import get_ai_response_cache
from app.utils.operational_errors import RECOVERABLE_ERRORS
logger = logging.getLogger(__name__)
_ai_response_cache = get_ai_response_cache()
LEGACY_BASE_PROMPT = """你是一个专业的业务助手，服务于使用 XCAGI 系统的用户。
你的职责：
1. 友好、专业地回答用户问题
2. 协助用户处理发货单、产品、客户等业务
3. 提供清晰、简洁的回答
```

```
4. 如果不确定，请诚实地告知用户
XCAGI 系统主要功能：
- 发货单生成和管理
- 产品和客户管理
- 订单处理
- 文件上传和导出
- 数据查询和统计"""
def _make_ai_response_cache_key(message: str, context_hash: str = "") -> str:
import hashlib
return hashlib.sha256(
f"ai_response:v1:{context_hash}:{message.strip().lower()}".encode()
).hexdigest()
def _coerce_int(value: Any) -> int:
try:
return int(value or 0)
except (TypeError, ValueError):
return 0
def _build_llm_trace(provider: Any, result: dict[str, Any], latency_ms: float) -> dict[str, Any]:
from app.infrastructure.billing.model_usage import estimate_llm_cost_units
adapter = getattr(provider, "_adapter", None)
provider_id = str(getattr(provider, "provider_id", "") or "")
provider_name = str(
getattr(adapter, "provider_name", "")
or getattr(provider, "provider_name", "")
or provider_id
)
model = str(
result.get("model")
or getattr(adapter, "model_name", "")
or getattr(provider, "model_name", "")
or ""
)
usage = result.get("usage") if isinstance(result.get("usage"), dict) else {}
prompt_tokens = _coerce_int(usage.get("prompt_tokens"))
completion_tokens = _coerce_int(usage.get("completion_tokens"))
total_tokens = _coerce_int(usage.get("total_tokens"))
cost_units = estimate_llm_cost_units(
prompt_tokens=prompt_tokens,
completion_tokens=completion_tokens,
total_tokens=total_tokens,
)
return {
"provider_id": provider_id,
"provider": provider_name,
"model": model,
"prompt_tokens": prompt_tokens,
"completion_tokens": completion_tokens,
"total_tokens": total_tokens,
"latency_ms": round(float(latency_ms), 2),
"cost_units": cost_units,
```

```
"billing_status": "metered" if cost_units else "unmetered",
"billing_source": "estimated_token_units",
}
class ApiMixin(NeuroEventPublisherMixin):
async def _get_deepseek_async_client(self):
import asyncio
import httpx
loop = asyncio.get_running_loop()
if self._deepseek_async_loop is not loop:
if self._deepseek_async_client is not None:
try:
await self._deepseek_async_client.aclose()
except RECOVERABLE_ERRORS:
logger.debug("suppressed exception", exc_info=True)
self._deepseek_async_client = None
self._deepseek_async_loop = loop
self._deepseek_async_client = httpx.AsyncClient(
timeout=httpx.Timeout(30.0, connect=10.0),
limits=httpx.Limits(max_keepalive_connections=10, max_connections=30),
)
return self._deepseek_async_client
async def _call_ai_offline(
self,
message: str,
context: ConversationContext,
intent_result: dict[str, Any],
) -> dict[str, Any]:
final_intent = intent_result.get("final_intent") or intent_result.get("primary_intent")
if final_intent and final_intent != "unk":
reply = (
f"当前为离线模式，已识别意图：{final_intent}。"
"如需更强的开放问答能力，可在系统设置切换到在线模式。"
)
else:
reply = (
"当前为离线模式，我可以继续处理开单、查询、打印等本地可执行流程。"
"如果你希望进行复杂问答，请在系统设置切换到在线模式。"
)
self.add_to_history(context.user_id, "user", message)
self.add_to_history(context.user_id, "assistant", reply)
return {
"text": reply,
"action": "offline_response",
"data": {
"intent": intent_result,
"mode": "offline",
},
}
async def call_llm_api(
self,
```

```
messages: list[dict[str, str]],
temperature: float = 0.7,
max_tokens: int = 2000,
**kwargs,
) -> dict[str, Any] | None:
t0 = time.perf_counter()
try:
from app.infrastructure.llm.providers.registry import get_active_provider
provider = get_active_provider(conversation_service=self)
if provider is None:
logger.error(" 无可用的 LLM Provider（检查 LLM_ROUTING_ORDER / 密钥）")
return None
logger.info(" [LLM] provider=%s", provider.provider_id)
result = await provider.chat_completion(
messages=messages,
temperature=temperature,
max_tokens=max_tokens,
**kwargs,
)
if result:
latency_ms = (time.perf_counter() - t0) * 1000.0
trace = _build_llm_trace(provider, result, latency_ms)
result["_xcagi_trace"] = trace
self._last_llm_trace = trace
try:
from app.infrastructure.billing.model_usage import (
record_model_usage,
)
record_model_usage(
provider_id=trace["provider_id"],
provider=trace["provider"],
model=trace["model"],
prompt_tokens=trace["prompt_tokens"],
completion_tokens=trace["completion_tokens"],
total_tokens=trace["total_tokens"],
cost_units=trace["cost_units"],
billing_status=trace["billing_status"],
billing_source=trace["billing_source"],
source="conversation_service",
user_id=str(
getattr(
getattr(self, "modstore_adapter", None),
"user_id",
"",
)
or ""
),
)
except RECOVERABLE_ERRORS:
logger.debug("record_model_usage failed", exc_info=True)
```

```
try:
from app.neuro_bus.application_neuro_bridge import (
neuro_notify_ai_model_roundtrip,
)
neuro_notify_ai_model_roundtrip(
model=trace["model"] or trace["provider_id"],
latency_ms=latency_ms,
token_count=trace["total_tokens"],
user_id=str(
getattr(getattr(self, "modstore_adapter", None), "user_id", "") or ""
),
)
except RECOVERABLE_ERRORS:
pass
return result
except RECOVERABLE_ERRORS as e:
logger.error(" LLM API调用异常: %s", e, exc_info=True)
return None
async def _call_deepseek_legacy(
self,
messages: list[dict[str, str]],
temperature: float = 0.7,
max_tokens: int = 2000,
**kwargs,
) -> dict[str, Any] | None:
import httpx
if not self.api_key:
logger.error("DeepSeek API Key 未配置")
return None
headers = {"Authorization": f"Bearer {self.api_key}", "Content-Type": "application/json"}
payload = {
"model": self.model,
"messages": messages,
"temperature": temperature,
"max_tokens": max_tokens,
**kwargs,
}
t0 = time.perf_counter()
try:
client = await self._get_deepseek_async_client()
response = await client.post(
self.api_url,
headers=headers,
json=payload,
)
response.raise_for_status()
result = response.json()
if result.get("choices") and len(result["choices"]) > 0:
latency_ms = (time.perf_counter() - t0) * 1000.0
usage = result.get("usage") if isinstance(result.get("usage"), dict) else {}
```

```
prompt_tokens = _coerce_int(usage.get("prompt_tokens"))
completion_tokens = _coerce_int(usage.get("completion_tokens"))
total_tokens = _coerce_int(usage.get("total_tokens"))
try:
from app.infrastructure.billing.model_usage import (
estimate_llm_cost_units,
record_model_usage,
)
cost_units = estimate_llm_cost_units(
prompt_tokens=prompt_tokens,
completion_tokens=completion_tokens,
total_tokens=total_tokens,
)
record_model_usage(
provider_id="deepseek-legacy",
provider="deepseek",
model=str(self.model or ""),
prompt_tokens=prompt_tokens,
completion_tokens=completion_tokens,
total_tokens=total_tokens,
cost_units=cost_units,
billing_status="metered" if cost_units else "unmetered",
billing_source="estimated_token_units",
source="conversation_service.deepseek_legacy",
)
except RECOVERABLE_ERRORS:
logger.debug("record_model_usage failed", exc_info=True)
try:
from app.neuro_bus.application_neuro_bridge import (
neuro_notify_ai_model_roundtrip,
)
neuro_notify_ai_model_roundtrip(
model=self.model,
latency_ms=latency_ms,
token_count=total_tokens,
user_id="",
)
except RECOVERABLE_ERRORS:
logger.debug("neuro_notify_ai_model_roundtrip skipped", exc_info=True)
return cast("dict[str, Any] | None", result)
logger.warning("DeepSeek API 返回空响应：%s", result)
return None
except httpx.HTTPError as e:
logger.error("DeepSeek API 请求失败：%s", e)
return None
except RECOVERABLE_ERRORS as e:
logger.error("调用 DeepSeek API 异常：%s", e)
return None
call_deepseek_api = call_llm_api
def _maybe_attach_kitten_web(
```

```
self,
conv_context: ConversationContext,
result: dict[str, Any],
) -> dict[str, Any]:
req = (conv_context.metadata or {}).get("request_context") or {}
if not req.get("kitten_analyzer") or not req.get("kitten_web_search"):
return result
payload = result.get("data")
if not isinstance(payload, dict):
payload = {}
result["data"] = payload
payload["web_search_results"] = list(req.get("web_search_results") or [])
meta = req.get("web_search_meta")
if meta:
payload["web_search_meta"] = meta
err = req.get("web_search_error")
if err:
payload["web_search_error"] = err
return result
async def _execute_or_generate_response(
self,
message: str,
intent_result: dict[str, Any],
conv_context: ConversationContext,
user_id: str,
) -> dict[str, Any]:
final_intent = intent_result.get("final_intent") or intent_result.get("primary_intent")
slots = intent_result.get("slots", {})
pending = self.confirmation_service.get_pending_intent(user_id)
if pending:
slots = self.confirmation_service.merge_slots(user_id, slots)
check_result = self.confirmation_service.check_and_build_prompt(
{
"final_intent": final_intent,
"slots": slots,
}
)
if check_result["status"] == "missing_slots":
self.confirmation_service.set_pending_intent(user_id, check_result["pending_data"])
return {
"text": check_result["question"],
"action": "slot_fill",
"data": {
"intent": final_intent,
"slots": slots,
"missing_slots": check_result["missing_slots"],
"pending_data": check_result["pending_data"],
},
}
tool_key = intent_result.get("tool_key")
```

```
if tool_key:
return self._build_tool_call_response(
tool_key, slots, intent_result, user_id, check_result
)
if intent_result.get("ai_mode") == "offline":
return await self._call_ai_offline(message, conv_context, intent_result)
return await self._call_ai(message, conv_context, intent_result)
def _build_tool_call_response(
self,
tool_key: str,
slots: dict[str, Any],
intent_result: dict[str, Any],
user_id: str,
check_result: dict[str, Any],
) -> dict[str, Any]:
tool_action_texts = {
"shipment_generate": f"正在为 {slots.get('unit_name', slots.get('keyword', '该客户'))} 生成发货单",
"products": f"正在查询 {slots.get('unit_name', slots.get('keyword', '该产品'))} 的产品信息",
"customers": "正在查询客户信息",
"shipments": "正在查询发货记录",
"print_label": "正在处理标签打印",
"wechat_send": "正在发送微信消息",
"upload_file": "正在上传文件",
"materials": "正在查询原材料库存",
"shipment_template": "正在查询发货单模板",
"template_extract": "正在提取模板结构",
"business_docking": "正在执行业务对接模板提取",
"template_preview": "正在查询模板预览",
"shipment_records": "正在查询出货记录",
"wechat": "正在处理微信联系人能力",
"printer_list": "正在查询打印机配置",
"settings": "正在读取系统设置",
"tools_table": "正在加载工具能力表",
"other_tools": "正在查询其他工具能力",
"ai_ecosystem": "正在查询AI生态能力",
"excel_decompose": "正在分解Excel模板",
"excel_analyzer": "正在分析Excel结构",
"show_images": "正在查看图片",
"show_videos": "正在查看视频",
}
action_text = tool_action_texts.get(tool_key, f"正在处理工具调用：{tool_key}")
habit_suggestion = self._check_habit_suggestion(user_id, tool_key, slots)
if habit_suggestion:
action_text = f"{action_text} {habit_suggestion}"
self.confirmation_service.set_pending_intent(
user_id,
{
"intent": intent_result.get("final_intent") or intent_result.get("primary_intent"),
"slots": slots,
"missing_slots": (
```

```
check_result.get("missing_slots", [])
if check_result.get("status") == "missing_slots"
else []
),
},
)
return {
"text": f"好的，{action_text}...",
"action": "tool_call",
"data": {
"tool_key": tool_key,
"intent": intent_result.get("final_intent") or intent_result.get("primary_intent"),
"slots": slots,
"hints": intent_result.get("intent_hints", []),
"habit_suggestion": habit_suggestion,
},
}
def _build_system_prompt_with_persona(
self,
user_id: str,
message: str,
history: list[dict],
industry: str,
context_prompt: str,
) -> str:
if not getattr(self, "persona_service", None):
return LEGACY_BASE_PROMPT + ("\n\n" + context_prompt if context_prompt else "")
return self.persona_service.build_prompt(None, context_prompt)
async def _call_ai(
self, message: str, context: ConversationContext, intent_result: dict[str, Any]
) -> dict[str, Any]:
context_hash = self._metadata_cache_hash(context.metadata)
cache_key = _make_ai_response_cache_key(message, context_hash)
cached_response = _ai_response_cache.get(cache_key)
if cached_response:
logger.debug("返回缓存的 AI 响应")
current_model = (
getattr(self.llm_adapter, "model_name", None)
if hasattr(self, "llm_adapter") and self.llm_adapter
else self.model
)
return {
"text": cached_response,
"action": "ai_response",
"data": {"model": current_model, "cached": True, "intent": intent_result},
}
context_prompt = self._build_context_prompt(context)
if getattr(self, "persona_service", None):
_req_ctx = (context.metadata or {}).get("request_context") or {}
system_prompt, _persona_params = await self.persona_service.build_prompt_from_message(
```

```
user_id=context.user_id,
message=message,
history=context.conversation_history or [],
industry=_req_ctx.get("industry", "通用"),
context_prompt=context_prompt,
)
else:
system_prompt = LEGACY_BASE_PROMPT + ("\n\n" + context_prompt if context_prompt else "")
messages = [{"role": "system", "content": system_prompt}]
if context.conversation_history:
messages.extend(context.conversation_history[-10:])
messages.append({"role": "user", "content": message})
if hasattr(self, "modstore_adapter") and self.modstore_adapter:
mode_tag = "平台"
provider_info = (
f"modstore:{self.modstore_adapter.default_provider}/"
f"{self.modstore_adapter.default_model}"
)
elif hasattr(self, "llm_adapter") and self.llm_adapter and self.llm_adapter.is_configured:
mode_tag = "直连"
provider_info = f"{self.llm_adapter.provider_name}/{self.llm_adapter.model_name}"
else:
mode_tag = "降级"
provider_info = f"DeepSeek/{self.model}"
logger.info("准备调用 LLM API [%s]: %s", mode_tag, provider_info)
response = await self.call_llm_api(messages)
logger.info(
"LLM API 响应 [%s %s]: %s...",
mode_tag,
provider_info,
str(response)[:150] if response else "None",
)
if response and response.get("choices"):
ai_reply = response["choices"][0]["message"]["content"]
self.add_to_history(context.user_id, "user", message)
self.add_to_history(context.user_id, "assistant", ai_reply)
_ai_response_cache.set(cache_key, ai_reply)
llm_trace = dict(response.get("_xcagi_trace") or {})
if llm_trace:
current_model = str(llm_trace.get("model") or self.model)
current_provider = str(
llm_trace.get("provider") or llm_trace.get("provider_id") or ""
)
elif hasattr(self, "modstore_adapter") and self.modstore_adapter:
current_model = f"modstore:{self.modstore_adapter.default_model}"
current_provider = "modstore-platform"
elif (
hasattr(self, "llm_adapter") and self.llm_adapter and self.llm_adapter.is_configured
):
current_model = self.llm_adapter.model_name
```

```
current_provider = self.llm_adapter.provider_name
else:
current_model = self.model
current_provider = "deepseek-legacy"
return {
"text": ai_reply,
"action": "ai_response",
"data": {
"model": current_model,
"provider": current_provider,
"llm_trace": llm_trace,
"usage": response.get("usage", {}),
"intent": intent_result,
},
}
else:
fallback_reply = '抱歉，我暂时无法理解您的需求。您可以：\n• 重新描述您的问题\n• 使用更简单的语句\n• 联系人工客服获取帮助\n\n如果需要帮助，请说"帮助"或"功能介绍"。'
self.add_to_history(context.user_id, "user", message)
self.add_to_history(context.user_id, "assistant", fallback_reply)
return {"text": fallback_reply, "action": "fallback", "data": {"intent": intent_result}}
// 文件：services/conversation/context.py
import logging
import time
from dataclasses import dataclass, field
from typing import Any, cast
from app.utils.operational_errors import RECOVERABLE_ERRORS
logger = logging.getLogger(__name__)
@dataclass
class ConversationContext:
user_id: str
conversation_history: list[dict[str, str]] = field(default_factory=list)
current_file: str | None = None
last_action: str | None = None
metadata: dict[str, Any] = field(default_factory=dict)
created_at: float = field(default_factory=time.time)
updated_at: float = field(default_factory=time.time)
current_intent: str | None = None
current_tool_key: str | None = None
intent_hints: list[str] = field(default_factory=list)
pending_confirmation: dict[str, Any] | None = None
last_intent_result: dict[str, Any] | None = None
class ContextMixin:
def get_context(self, user_id: str) -> ConversationContext | None:
return cast("ConversationContext | None", self.contexts.get(user_id))
def create_context(self, user_id: str) -> ConversationContext:
context = ConversationContext(user_id=user_id)
self.contexts[user_id] = context
logger.info("为用户 %s 创建新的对话上下文", user_id)
return context
```

```
def update_context(self, user_id: str, **kwargs) -> ConversationContext | None:
context = self.contexts.get(user_id)
if not context:
return None
for key, value in kwargs.items():
if hasattr(context, key):
setattr(context, key, value)
context.updated_at = time.time()
return cast("ConversationContext | None", context)
def set_pending_confirmation(self, user_id: str, confirmation_data: dict[str, Any]) -> bool:
context = self.contexts.get(user_id)
if not context:
context = self.create_context(user_id)
context.pending_confirmation = confirmation_data
context.updated_at = time.time()
return True
def _get_or_create_context(
self, user_id: str, context: dict[str, Any] | None
) -> ConversationContext:
conv_context = self.get_context(user_id)
if not conv_context:
conv_context = self.create_context(user_id)
enriched = self._enrich_context_with_kitten_business_snapshot(context)
self._apply_request_context(conv_context, enriched)
return conv_context
async def _get_or_create_context_async(
self,
user_id: str,
context: dict[str, Any] | None,
message: str,
) -> ConversationContext:
conv_context = self.get_context(user_id)
if not conv_context:
conv_context = self.create_context(user_id)
enriched = self._enrich_context_with_kitten_business_snapshot(context)
enriched = await self._enrich_kitten_web_search_if_needed(enriched, message, user_id)
self._apply_request_context(conv_context, enriched)
return conv_context
def _update_context_from_intent(
self, conv_context: ConversationContext, intent_result: dict[str, Any]
) -> None:
conv_context.current_intent = intent_result.get("final_intent") or intent_result.get(
"primary_intent"
)
conv_context.current_tool_key = intent_result.get("tool_key")
conv_context.intent_hints = intent_result.get("intent_hints", [])
conv_context.last_intent_result = intent_result
def _enrich_context_with_kitten_business_snapshot(
self, context: dict[str, Any] | None
) -> dict[str, Any] | None:
```

```
if not isinstance(context, dict):
return context
if not context.get("kitten_analyzer"):
return context
out = dict(context)
if out.get("kitten_include_business_db"):
try:
from app.services.kitten_business_snapshot import (
build_kitten_business_snapshot,
)
out["kitten_business_snapshot"] = build_kitten_business_snapshot()
except RECOVERABLE_ERRORS as exc:
logger.warning("kitten business snapshot build failed: %s", exc)
out["kitten_business_snapshot"] = {
"success": False,
"text": f"【业务数据库快照】生成失败：{exc}",
"stats": {},
}
else:
out.pop("kitten_business_snapshot", None)
return out
async def _enrich_kitten_web_search_if_needed(
self,
context: dict[str, Any] | None,
message: str,
user_id: str,
) -> dict[str, Any] | None:
if not isinstance(context, dict):
return context
if not context.get("kitten_analyzer") or not context.get("kitten_web_search"):
return context
try:
from app.infrastructure.web_search import kitten_web_search
result = await kitten_web_search(message.strip(), user_key=user_id or "anonymous")
except RECOVERABLE_ERRORS as exc:
logger.warning("kitten web search: %s", exc)
out = dict(context)
out["web_search_results"] = []
out["web_search_error"] = str(exc)
return out
out = dict(context)
out["web_search_results"] = result.get("hits") if result.get("success") else []
if not result.get("success"):
out["web_search_error"] = result.get("message") or "search failed"
else:
out.pop("web_search_error", None)
out["web_search_meta"] = {
"provider": result.get("provider"),
"query": result.get("query"),
}
```

```
return out
def _apply_request_context(
self,
conv_context: ConversationContext,
ctx: dict[str, Any] | None,
) -> None:
if ctx is None:
return
if not ctx:
return
prev = (conv_context.metadata or {}).get("request_context") or {}
merged: dict[str, Any] = {**prev, **ctx}
if ctx.get("kitten_analyzer") and ctx.get("has_dataset") is False:
merged.pop("kitten_dataset", None)
elif "kitten_dataset" in ctx:
if ctx["kitten_dataset"]:
merged["kitten_dataset"] = self._sanitize_kitten_dataset(ctx["kitten_dataset"])
else:
merged.pop("kitten_dataset", None)
elif prev.get("kitten_dataset"):
merged["kitten_dataset"] = prev["kitten_dataset"]
if ctx.get("kitten_analyzer"):
if ctx.get("kitten_include_business_db") and ctx.get("kitten_business_snapshot"):
merged["kitten_business_snapshot"] = self._sanitize_kitten_business_snapshot(
ctx["kitten_business_snapshot"]
)
else:
merged.pop("kitten_business_snapshot", None)
if not ctx.get("kitten_web_search"):
merged.pop("web_search_results", None)
merged.pop("web_search_error", None)
merged.pop("web_search_meta", None)
if "web_search_results" in ctx:
merged["web_search_results"] = self._sanitize_web_search_results(
ctx.get("web_search_results")
)
if ctx.get("web_search_error"):
merged["web_search_error"] = str(ctx.get("web_search_error"))[:500]
elif "web_search_error" in merged:
merged.pop("web_search_error", None)
meta = ctx.get("web_search_meta")
if isinstance(meta, dict) and meta:
merged["web_search_meta"] = {
k: str(v)[:200] for k, v in meta.items() if v is not None
}
elif "web_search_meta" in merged:
merged.pop("web_search_meta", None)
conv_context.metadata.setdefault("request_context", {})
conv_context.metadata["request_context"] = merged
conv_context.updated_at = time.time()
```

```
def clear_context(self, user_id: str) -> bool:
if user_id in self.contexts:
del self.contexts[user_id]
logger.info("已清除用户 %s 的对话上下文", user_id)
return True
return False
def get_all_contexts(self) -> dict[str, ConversationContext]:
return cast("dict[str, ConversationContext]", self.contexts.copy())
def cleanup_old_contexts(self, max_age_seconds: int = 3600) -> int:
current_time = time.time()
to_remove = []
for user_id, context in self.contexts.items():
if current_time - context.updated_at > max_age_seconds:
to_remove.append(user_id)
for user_id in to_remove:
del self.contexts[user_id]
if to_remove:
logger.info("清理了 %s 个过期的对话上下文", len(to_remove))
return len(to_remove)
// 文件：services/conversation/llm_adapter.py
import logging
import os
from abc import ABC, abstractmethod
from typing import Any, Dict, List, Optional, cast
import httpx
logger = logging.getLogger(__name__)
class BaseLLMAdapter(ABC):
@abstractmethod
async def chat_completion(
self,
messages: List[Dict[str, str]],
temperature: float = 0.7,
max_tokens: int = 2000,
**kwargs,
) -> Dict[str, Any]:
pass
@abstractmethod
async def stream_chat_completion(self, messages: List[Dict[str, str]], **kwargs):
pass
@property
@abstractmethod
def provider_name(self) -> str:
pass
@property
@abstractmethod
def model_name(self) -> str:
pass
class OpenAICompatibleAdapter(BaseLLMAdapter):
PROVIDER_DEFAULT_URLS: Dict[str, str] = {
```

```
"xcauto": "https://xiu-ci.com/v1",
"xiuci": "https://xiu-ci.com/v1",
"deepseek": "https://api.deepseek.com",
"xiaomi": "https://token-plan-cn.xiaomimimo.com",
"openai": "https://api.openai.com",
"siliconflow": "https://api.siliconflow.cn",
"groq": "https://api.groq.com/openai",
"together": "https://api.together.xyz",
"openrouter": "https://openrouter.ai/api",
"dashscope": "https://dashscope.aliyuncs.com/compatible-mode",
"moonshot": "https://api.moonshot.cn",
"minimax": "https://api.minimaxi.com",
"doubao": "https://ark.cn-beijing.volces.com/api/v3",
"wenxin": "https://qianfan.baidubce.com/v2",
"hunyuan": "https://api.hunyuan.cloud.tencent.com/v1",
"zhipu": "https://open.bigmodel.cn/api/paas/v4",
"xunfei": "https://spark-api-open.xf-yun.com/v1",
"yi": "https://api.lingyiwanwu.com/v1",
"stepfun": "https://api.stepfun.com/v1",
"baichuan": "https://api.baichuan-ai.com/v1",
"sensetime": "https://api.sensenova.cn/compatible-mode/v1",
}
DEFAULT_MODELS: Dict[str, str] = {
"xcauto": "xcauto-account",
"xiuci": "xcauto-account",
"xiaomi": "mimo-v2.5-pro",
"deepseek": "deepseek-chat",
"openai": "gpt-4o-mini",
"siliconflow": "Qwen/Qwen2.5-7B-Instruct",
"groq": "llama-3.3-70b-versatile",
"together": "meta-llama/Llama-3-70b-chat-hf",
"openrouter": "openai/gpt-3.5-turbo",
"dashscope": "qwen-plus",
"moonshot": "moonshot-v1-8k",
"minimax": "MiniMax-Text-01",
"doubao": "ep-20250515143000-l6zqx",
"wenxin": "ernie-speed-128k",
"hunyuan": "hunyuan-lite",
"zhipu": "glm-4-flash",
"xunfei": "spark-lite",
"yi": "yi-lightning",
"stepfun": "step-1-8k",
"baichuan": "Baichuan2-Turbo",
"sensetime": "SenseChat-5",
}
ENV_KEY_MAPPING: Dict[str, List[str]] = {
"xcauto": ["XCAUTO_API_KEY", "XCAUTO_PAT", "XIUCI_API_KEY", "OPENAI_API_KEY"],
"xiuci": ["XCAUTO_API_KEY", "XCAUTO_PAT", "XIUCI_API_KEY", "OPENAI_API_KEY"],
"xiaomi": ["XIAOMI_API_KEY", "MIMO_API_KEY", "XIAOMI_MIMO_API_KEY"],
"deepseek": ["DEEPSEEK_API_KEY"],
```

```
"openai": ["OPENAI_API_KEY"],
"anthropic": ["ANTHROPIC_API_KEY"],
"google": ["GEMINI_API_KEY", "GOOGLE_API_KEY"],
"siliconflow": ["SILICONFLOW_API_KEY"],
"groq": ["GROQ_API_KEY"],
"together": ["TOGETHER_API_KEY"],
"openrouter": ["OPENROUTER_API_KEY"],
"dashscope": ["DASHSCOPE_API_KEY"],
"moonshot": ["MOONSHOT_API_KEY"],
"minimax": ["MINIMAX_API_KEY"],
"doubao": ["DOUBAO_API_KEY", "ARK_API_KEY"],
"wenxin": ["WENXIN_API_KEY", "QIANFAN_API_KEY", "BAIDU_QIANFAN_API_KEY"],
"hunyuan": ["HUNYUAN_API_KEY", "TENCENT_HUNYUAN_API_KEY"],
"zhipu": ["ZHIPU_API_KEY", "BIGMODEL_API_KEY"],
"xunfei": ["XUNFEI_API_KEY", "SPARK_API_KEY"],
"yi": ["YI_API_KEY", "LINGYIWANWU_API_KEY"],
"stepfun": ["STEPFUN_API_KEY"],
"baichuan": ["BAICHUAN_API_KEY"],
"sensetime": ["SENSETIME_API_KEY", "SENSENOVA_API_KEY"],
}
def __init__(
self, provider: str = "xiaomi", model: str = None, api_key: str = None, base_url: str = None
):
self.provider = provider.lower().strip()
self._api_key = api_key or self._resolve_api_key(self.provider)
self._base_url = (
base_url.rstrip("/")
if base_url
else self.PROVIDER_DEFAULT_URLS.get(self.provider, "https://api.openai.com")
)
self._model = model or self.DEFAULT_MODELS.get(self.provider, "gpt-3.5-turbo")
self._client: Optional[httpx.AsyncClient] = None
self._stream_client: Optional[httpx.AsyncClient] = None
logger.info(
"初始化LLM适配器: %s/%s @ %s (Key长度: %s)",
self.provider,
self._model,
self._base_url,
len(self._api_key or ""),
)
def _resolve_api_key(self, provider: str) -> Optional[str]:
env_names = self.ENV_KEY_MAPPING.get(provider, [f"{provider.upper()}_API_KEY"])
for env_name in env_names:
key = os.environ.get(env_name, "").strip()
if key:
logger.debug("从环境变量 %s 读取到API Key", env_name)
return key
logger.warning("未找到 %s 的API Key (已检查: %s)", provider, env_names)
return None
@property
```

```
def provider_name(self) -> str:
return self.provider
@property
def model_name(self) -> str:
return self._model
@property
def is_configured(self) -> bool:
return bool(self._api_key)
async def _get_client(self) -> httpx.AsyncClient:
if self._client is None or self._client.is_closed:
self._client = httpx.AsyncClient(
timeout=httpx.Timeout(30.0, connect=10.0),
limits=httpx.Limits(max_keepalive_connections=10, max_connections=30),
)
return self._client
async def _get_stream_client(self) -> httpx.AsyncClient:
if self._stream_client is None or self._stream_client.is_closed:
self._stream_client = httpx.AsyncClient(
timeout=httpx.Timeout(connect=15.0, read=300.0, write=60.0, pool=30.0),
limits=httpx.Limits(max_keepalive_connections=200, max_connections=1000),
)
return self._stream_client
def _normalize_base_url(self) -> str:
url = self._base_url.rstrip("/")
if any(url.endswith(f"/v{i}") for i in range(1, 5)):
return url
return f"{url}/v1"
async def chat_completion(
self,
messages: List[Dict[str, str]],
temperature: float = 0.7,
max_tokens: int = 2000,
**kwargs,
) -> Dict[str, Any]:
if not self._api_key:
raise ValueError(f"[{self.provider}] API Key未配置")
base_url = self._normalize_base_url()
url = f"{base_url}/chat/completions"
payload: Dict[str, Any] = {
"model": self._model,
"messages": messages,
"temperature": temperature,
"max_tokens": max_tokens,
**kwargs,
}
headers = {"Authorization": f"Bearer {self._api_key}", "Content-Type": "application/json"}
logger.debug(
"调用 [%s/%s] messages=%s, temp=%s, max_tokens=%s",
self.provider,
self._model,
```

```
len(messages),
temperature,
max_tokens,
)
client = await self._get_client()
response = await client.post(url, headers=headers, json=payload)
response.raise_for_status()
result = response.json()
logger.debug(
"[%s] 响应成功, choices=%s, usage=%s",
self.provider,
len(result.get("choices", [])),
result.get("usage", {}),
)
return cast("dict[str, Any]", result)
async def stream_chat_completion(
self,
messages: List[Dict[str, str]],
temperature: float = 0.7,
max_tokens: int = 2000,
**kwargs,
):
if not self._api_key:
raise ValueError(f"[{self.provider}] API Key未配置")
base_url = self._normalize_base_url()
url = f"{base_url}/chat/completions"
payload: Dict[str, Any] = {
"model": self._model,
"messages": messages,
"temperature": temperature,
"max_tokens": max_tokens,
"stream": True,
"stream_options": {"include_usage": True},
**kwargs,
}
headers = {"Authorization": f"Bearer {self._api_key}", "Content-Type": "application/json"}
logger.info("启动流式请求 [%s/%s]", self.provider, self._model)
client = await self._get_stream_client()
async with client.stream("POST", url, headers=headers, json=payload) as response:
response.raise_for_status()
async for line in response.aiter_lines():
if line.startswith("data: "):
yield line[6:]
async def close(self):
if self._client and not self._client.is_closed:
await self._client.aclose()
if self._stream_client and not self._stream_client.is_closed:
await self._stream_client.aclose()
def __repr__(self) -> str:
configured = "" if self.is_configured else ""
```

```
return (
f"<OpenAICompatibleAdapter {configured} "
f"provider={self.provider}, model={self._model}, "
f"key={'*' * min(len(self._api_key or ''), 8)}>"
)
// 文件：services/conversation/manager.py
import logging
import os
import time
from typing import Any, cast
from app.services.conversation.api import ApiMixin
from app.services.conversation.context import ContextMixin, ConversationContext
from app.services.conversation.handlers import HandlersMixin
from app.services.conversation.intent import IntentMixin
from app.services.conversation.prompts import PromptsMixin
from app.utils.operational_errors import RECOVERABLE_ERRORS
logger = logging.getLogger(__name__)
class AIConversationService(
ContextMixin,
IntentMixin,
HandlersMixin,
PromptsMixin,
ApiMixin,
):
def __init__(self):
self.contexts: dict[str, ConversationContext] = {}
self.llm_adapter = None
self.modstore_adapter = None
llm_init_mode = "none"
try:
from app.services.conversation.modstore_adapter import (
create_modstore_adapter_from_env,
)
modstore = create_modstore_adapter_from_env()
if modstore and modstore.is_configured:
self.modstore_adapter = modstore
llm_init_mode = "platform"
logger.info(
" [优先级1] 启用修茁市场平台代理模式\n"
" 平台地址: %s\n"
" 默认模型: %s/%s\n"
" 用户ID: %s\n"
"  所有LLM请求将通过平台统一路由",
modstore.platform_url,
modstore.default_provider,
modstore.default_model,
modstore.user_id,
)
else:
```

```
logger.debug("未检测到修芷市场平台配置，尝试直连模式...")
if not self.modstore_adapter:
from app.infrastructure.llm.providers.credentials import (
resolve_default_chat_model,
resolve_default_openai_provider,
resolve_openai_env_credentials,
)
from app.services.conversation.llm_adapter import OpenAICompatibleAdapter
llm_provider = resolve_default_openai_provider()
llm_model = resolve_default_chat_model()
llm_api_key, llm_base_url = resolve_openai_env_credentials()
self.llm_adapter = OpenAICompatibleAdapter(
provider=llm_provider,
model=llm_model,
api_key=llm_api_key,
base_url=llm_base_url,
)
if self.llm_adapter.is_configured:
llm_init_mode = "direct"
logger.info(
" [优先级2] 启用直连模式: %s/%s (Key已配置)",
self.llm_adapter.provider_name,
self.llm_adapter.model_name,
)
else:
logger.warning(" 直连适配器已创建但 [%s] API Key未配置", llm_provider)
except RECOVERABLE_ERRORS as adapter_err:
logger.error(" LLM适配器初始化失败: %s", adapter_err)
self.llm_adapter = None
self.modstore_adapter = None
self.api_key = os.environ.get("DEEPSEEK_API_KEY", "")
if not self.api_key:
try:
from app.utils.path_utils import get_resource_path
config_path = get_resource_path("config", "deepseek_config.py")
if os.path.exists(config_path):
import importlib.util
spec = importlib.util.spec_from_file_location(
"xcagi_deepseek_config", config_path
)
if spec and spec.loader:
config_module = importlib.util.module_from_spec(spec)
spec.loader.exec_module(config_module)
self.api_key = getattr(config_module, "DEEPSEEK_API_KEY", "") or ""
except RECOVERABLE_ERRORS as e:
logger.warning("无法读取 resources/config/deepseek_config.py: %s", e)
if self.api_key and llm_init_mode == "none":
llm_init_mode = "legacy"
logger.info(
" [优先级3/降级] 使用旧版DeepSeek直连模式 (Key长度: %s)", len(self.api_key)
```

```
)
elif self.api_key:
logger.info("DeepSeek API Key 已配置（长度: %s）（作为降级备选)", len(self.api_key))
else:
logger.warning("DeepSeek API Key 未配置（降级路径不可用）")
from app.infrastructure.llm.providers.credentials import (
default_chat_completions_url,
resolve_default_chat_model,
)
self.api_url = default_chat_completions_url()
self.model = resolve_default_chat_model()
self._llm_mode = llm_init_mode
logger.info(" LLM初始化完成，运行模式: %s", llm_init_mode)
from app.services.deepseek_intent_service import HybridIntentWithDeepSeek
from app.services.intent_confirmation_service import get_confirmation_service
from app.services.intent_service import recognize_intents
from app.services.task_agent import get_task_agent
from app.services.unified_intent_recognizer import get_unified_intent_recognizer
from app.services.user_memory_service import get_user_memory_service
from app.services.user_preference_service import get_user_preference_service
use_distilled = os.environ.get("USE_DISTILLED_MODEL", "0") == "1"
if use_distilled:
logger.info("已启用蒸馏意图识别开关：USE_DISTILLED_MODEL=1")
self.intent_service = recognize_intents
self.online_intent_service = HybridIntentWithDeepSeek(
use_deepseek=True,
rule_priority=True,
confidence_threshold=0.6,
use_distilled=use_distilled,
)
self.offline_intent_service = HybridIntentWithDeepSeek(
use_deepseek=False,
rule_priority=True,
confidence_threshold=0.6,
use_distilled=True,
)
self.deepseek_intent_service = self.online_intent_service
self.unified_recognizer = get_unified_intent_recognizer()
self.confirmation_service = get_confirmation_service()
self.task_agent = get_task_agent()
self.user_memory = get_user_memory_service()
self.user_preference_service = get_user_preference_service()
self._deepseek_async_client: Any = None
self._deepseek_async_loop: Any = None
self.persona_service = None
def add_to_history(self, user_id: str, role: str, content: str) -> bool:
context = self.contexts.get(user_id)
if not context:
context = self.create_context(user_id)
context.conversation_history.append({"role": role, "content": content})
```

```
if len(context.conversation_history) > 20:
context.conversation_history = context.conversation_history[-20:]
context.updated_at = time.time()
return True
def add_intent_feedback(
self,
user_id: str,
message: str,
recognized_intent: str,
feedback: str,
corrected_intent: str | None = None,
slots: dict[str, Any] | None = None,
) -> None:
try:
self.user_memory.add_feedback(
user_id=user_id,
message=message,
recognized_intent=recognized_intent,
feedback=feedback,
corrected_intent=corrected_intent,
slots=slots or {},
)
except RECOVERABLE_ERRORS as e:
logger.error("添加意图反馈失败: %s", e)
def record_user_action(
self,
user_id: str,
intent: str,
slots: dict[str, Any],
message: str,
) -> None:
try:
self.user_memory.record_action(
user_id=user_id,
intent=intent,
slots=slots,
message=message,
)
except RECOVERABLE_ERRORS as e:
logger.error("记录用户操作失败: %s", e)
def apply_memory_preferences(
self, user_id: str, intent: str, slots: dict[str, Any]
) -> dict[str, Any]:
try:
return cast(
"dict[str, Any]", self.user_memory.apply_preference_to_slots(user_id, intent, slots)
)
except RECOVERABLE_ERRORS as e:
logger.error("应用用户偏好失败: %s", e)
return slots
```

```
def get_memory_similar_action(
self, user_id: str, intent: str, slots: dict[str, Any]
) -> dict[str, Any] | None:
try:
return cast(
"dict[str, Any] | None",
self.user_memory.get_similar_pattern(user_id, intent, slots),
)
except RECOVERABLE_ERRORS as e:
logger.error("获取相似操作失败: %s", e)
return None
def get_habit_suggestions(self, user_id: str) -> list[dict[str, Any]]:
try:
return cast("list[dict[str, Any]]", self.user_memory.get_habit_suggestions(user_id))
except RECOVERABLE_ERRORS as e:
logger.error("获取习惯建议失败: %s", e)
return []
def get_context_for_recognition(
self, user_id: str, conv_context: ConversationContext
) -> dict[str, Any]:
context = {
"user_id": user_id,
"current_intent": conv_context.current_intent,
"current_tool_key": conv_context.current_tool_key,
"last_intent": conv_context.current_intent,
"last_tool_key": conv_context.current_tool_key,
"last_slots": (
conv_context.last_intent_result.get("slots", {})
if conv_context.last_intent_result
else {}
),
"pending_confirmation": conv_context.pending_confirmation,
}
recent_actions = self.user_memory.get_recent_actions(user_id, limit=3)
if recent_actions:
context["recent_intents"] = [a.get("intent") for a in recent_actions]
preferences = self.user_memory.get_all_preferences(user_id)
if preferences:
context["user_preferences"] = preferences
return context
def _check_habit_suggestion(
self, user_id: str, current_intent: str, slots: dict[str, Any]
) -> str | None:
try:
habits = self.get_habit_suggestions(user_id)
for habit in habits:
actions = habit.get("actions", [])
confidence = habit.get("confidence", 0)
if confidence < 0.8:
continue
```

```
for action in actions:
if action.get("intent") == current_intent:
return f" 根据您的习惯，您可能还需要：{action.get('description', '')}"
except RECOVERABLE_ERRORS as e:
logger.error("检查习惯建议失败: %s", e)
return None
async def chat(
self,
user_id: str,
message: str,
context: dict[str, Any] | None = None,
source: str | None = None,
) -> dict[str, Any]:
try:
conv_context = await self._get_or_create_context_async(user_id, context, message)
intent_result = await self._recognize_intent(message, source, user_id, context)
intent_result = self._enhance_intent_slots(message, intent_result, user_id)
logger.info(
"[INTENT_RESULT] final_intent=%s, primary_intent=%s, tool_key=%s, slots=%s, intent_source=%s",
intent_result.get("final_intent"),
intent_result.get("primary_intent"),
intent_result.get("tool_key"),
intent_result.get("slots"),
intent_result.get("intent_source"),
)
self._update_context_from_intent(conv_context, intent_result)
if result := await self._handle_special_intents(
message, intent_result, conv_context, user_id
):
return self._maybe_attach_kitten_web(conv_context, result)
if result := await self._handle_pending_intent(
message, intent_result, conv_context, user_id
):
return self._maybe_attach_kitten_web(conv_context, result)
out = await self._execute_or_generate_response(
message, intent_result, conv_context, user_id
)
return self._maybe_attach_kitten_web(conv_context, out)
except RECOVERABLE_ERRORS as e:
logger.error("处理聊天消息失败：%s", e)
return {
"text": f"抱歉，处理消息时出现问题：{str(e)}",
"action": "error",
"data": {"message": str(e)},
}
_ai_conversation_service: AIConversationService | None = None
class _InMemoryPersonaRepository:
def __init__(self):
self._store: dict[str, Any] = {}
self._events: dict[str, list[dict]] = {}
```

```
async def find_by_user_id(self, user_id: str):
return self._store.get(user_id)
async def save(self, profile):
self._store[profile.user_id] = profile
return profile
async def delete(self, user_id: str) -> bool:
return self._store.pop(user_id, None) is not None
async def append_event(self, user_id: str, event_type: str, event_data: dict) -> None:
self._events.setdefault(user_id, []).append({"type": event_type, "data": event_data})
async def list_recent_events(self, user_id: str, limit: int = 20) -> list[dict]:
return self._events.get(user_id, [])[-limit:]
def _build_persona_repository():
try:
from app.infrastructure.persona.persona_repository_impl import PersonaRepositoryImpl
return PersonaRepositoryImpl()
except Exception as exc:
logger.warning("Persona 持久化仓储不可用，降级内存仓储: %s", exc)
return _InMemoryPersonaRepository()
def _create_persona_service():
from app.infrastructure.persona.embedding_client import EmbeddingClient
from app.infrastructure.persona.llm_client_adapter import PersonaLlmClient
from app.services.persona.axes_fuser import AxesFuser
from app.services.persona.embedding_inferencer import EmbeddingInferencer
from app.services.persona.identity_resolver import IdentityResolver
from app.services.persona.llm_inferencer import LlmInferencer
from app.services.persona.param_mapper import PersonaParamMapper
from app.services.persona.persona_service import PersonaService
from app.services.persona.prompt_builder import PersonaPromptBuilder
from app.services.persona.rapport_calculator import RapportCalculator
from app.services.persona.rule_inferencer import RuleInferencer
identity_resolver = IdentityResolver()
return PersonaService(
repo=_build_persona_repository(),
rule_inferencer=RuleInferencer(),
embedding_inferencer=EmbeddingInferencer(EmbeddingClient()),
llm_inferencer=LlmInferencer(PersonaLlmClient()),
axes_fuser=AxesFuser(),
rapport_calculator=RapportCalculator(),
identity_resolver=identity_resolver,
prompt_builder=PersonaPromptBuilder(identity_resolver),
param_mapper=PersonaParamMapper(),
)
def get_ai_conversation_service() -> AIConversationService:
global _ai_conversation_service
if _ai_conversation_service is None:
_ai_conversation_service = AIConversationService()
_ai_conversation_service.persona_service = _create_persona_service()
return _ai_conversation_service
def init_ai_conversation_service() -> AIConversationService:
global _ai_conversation_service
```

```
_ai_conversation_service = AIConversationService()
_ai_conversation_service.persona_service = _create_persona_service()
logger.info("AI 对话服务已初始化（已注入 PersonaService）")
return _ai_conversation_service
from app.neuro_bus.neuro_service_instrumentation import instrument_service_layer_class
instrument_service_layer_class(AIConversationService, "app.services.ai_conversation_service")
```